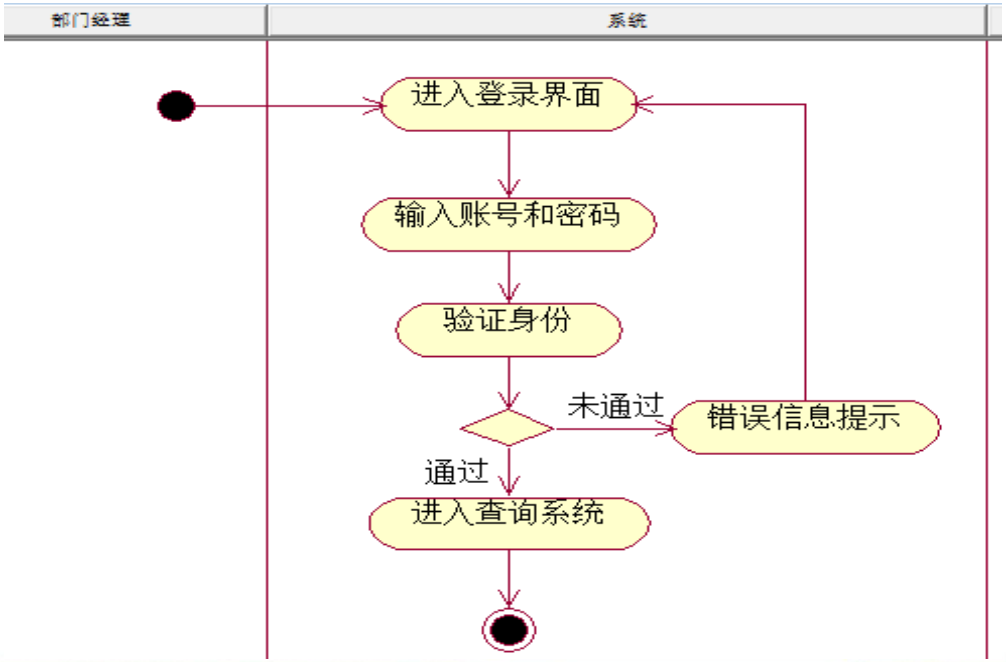
第11章 活动图和包图

- 程序中会出现大量的工作流和算法，这些复杂过程的步骤序列可以通过活动图来建模。顺序图也体现了过程的步骤序列，但是它偏重于对象，而活动图则偏重于操作。

- UML建模元素非常多，如何组织这些建模元素则通过包图来体现。

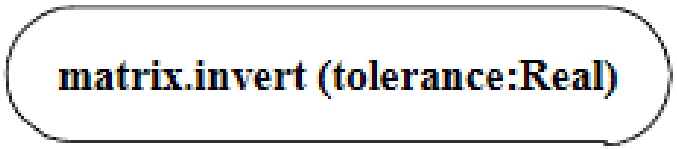
11.1 活动图概述-活动图概述

- 活动图是一种特殊形式的状态图，用于对计算流程和工作流程建模。
- 活动图中的状态表示计算过程中所处的各种状态，而不是普通对象的状态。
- 通常，活动图假定在整个计算处理的过程中没有外部事件引起的中断，否则，普通的状态图更适于描述这种情况。

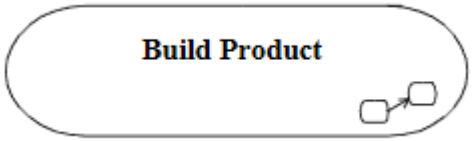


11.1 活动图概述-活动图概述

- 活动图包含活动状态（activity state）。活动状态表示过程中命令的执行或工作流程中活动的进行。与等待某一个事件发生的一般等待状态不同，活动状态等待计算处理工作的完成。当活动完成后，执行流程转入活动图中的下一个活动状态。当一个活动的前趋活动完成时，活动图中的完成转换被激发。
- 活动状态通常不会因为明确的事件而引起活动转换，但当包含它的活动转换时，活动状态会异常终止。



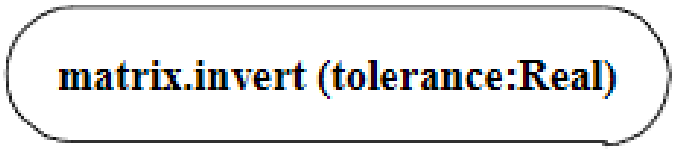
UML 1 活动状态表示



UML 1.4活动状态表示

11.1 活动图建模-活动图概述

- 活动图也可以包含动作状态(action state)，它与活动状态有些相似，但是它们是原子活动并且当它们处于活动状态时不允许发生转换。动作状态通常用于短的记帐操作。
  - 这里的“记账”不是指实际的会计活动，而是在软件和系统工程中，一种轻量级、快速完成且通常不会被打断的任务。例如，更新一个变量的值，检查一个条件是否满足，或者是发送一条消息通知。由于这些操作很短，不需要进一步的状态转换直到它们完成，所以把它们模型化为动作状态是合适的。



动作状态表示：也是圆角长方形

11.1 活动图建模-活动图概述

- UML 2.x的变化*:
  - 在UML 2.x中，活动图的表示方法进行了标准化，其中动作（Action）是一个基本构造块，它表示系统执行的原子操作；这意味着它不可再分，并且在动作执行过程中，不会发生状态转移。
  - 另一方面，活动（Activity）通常表示一个包含了多个动作和控制流程的工作流。活动节点（Activity Node）是指活动图中的任何节点，它包括动作节点、控制流节点（如决策节点和合并节点）、对象节点等等



Figure 15.4 ActivityNode notation

11.1 活动图建模-活动图概述

- 活动图可以包含并发线程的分叉（fork)控制。并发线程表示能被系统中的不同对象或者组织中的人并发执行的活动。
- 通常并发源于聚合(aggregation)，在聚合关系中每个对象有着它们自己的线程，这些线程可并发执行。并发活动可以同时执行也可以按任意顺序执行。
- 活动图不仅能够表达顺序流程控制还能够表达并发流程控制，如果排除了这一点，活动图很像一个传统的流程图。

11.1.2 活动图

- 活动图的各个步骤都是操作，尤其是来自状态模型的活动图。活动图的目标是显示复杂过程内部的各个步骤以及他们之间的顺序关系。UML建模中，活动图是活动视图的表示法。它包括一些方便的速记符号，这些符号实际上可以用于任何状态图，尽管活动图和状态图的混合表示法多数时候都很难看。
- 活动图用来描述系统或者某个业务的处理流程。例如，业务相关的工作流的描述、用例的事件流的描述、程序算法的描述等。

11.1.2活动图-模型元素

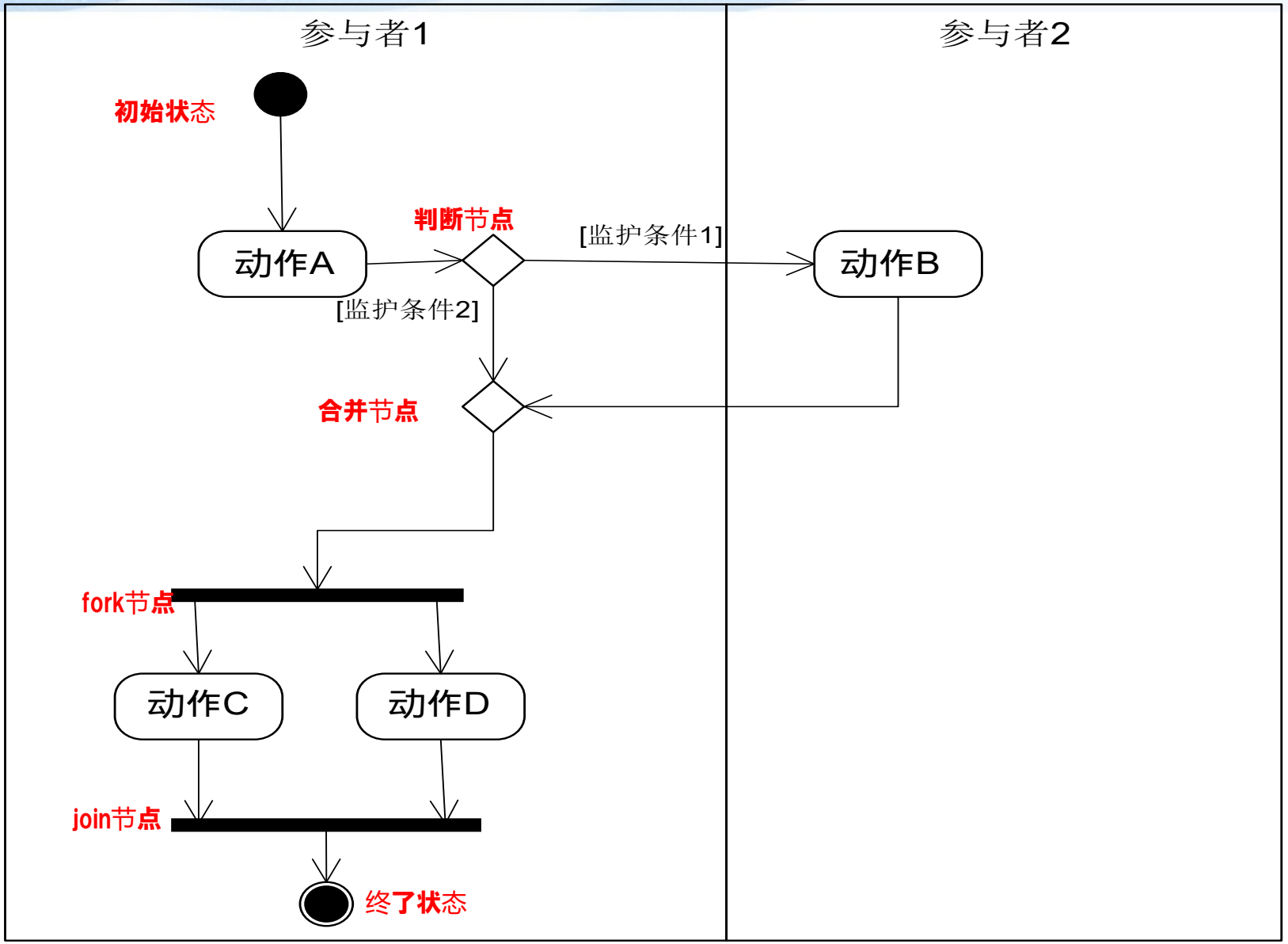
- 活动图用来描述系统或者某个业务的处理流程
  - 业务相关的工作流的描述
  - 用例的事件流的描述
  - 程序算法的描述
- 活动图由以下模型元素组成：
  - 活动，控制流，开始节点，终了节点，判断节点和监护条件， fork节点， join节点，分区（泳道）等。

11.1.2活动图-模型元素

- 活动
  - 相当于构成业务和处理的一个单位。用圆角长方形来表示。
- 控制流
  - 用箭头表示，连接两个活动，表示活动的先后顺序。
- 开始节点
  - 用实心圆圈表示一个流程的开始。
- 终了节点
  - 用实心圆圈外加一圆圈来表示一个流程的结束。
- 判断节点和监护条件
  - 判断节点用菱形框来表示，根据判断结果的不同，处理流向不同的分支。
  - 监护条件用来表示处理分支的条件。
- fork节点
  - 表示一个处理流程分割为多个并发流程。
- join节点
  - 表示对多个并发子流程进行同步。（※在UML2.0中，fork节点和join节点合称为同步条）
- 分区
  - 在UML1.X中也称为泳道。

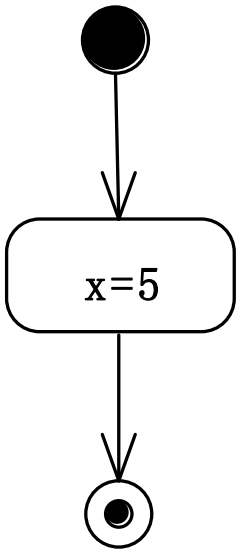
活动分区是具有某些共同特征的操作的活动分组。
分区通常对应于业务模型中的组织单位或业务参与者。
以泳道形式表示。

11.1.2活动图-模型元素



1. 控制流（flow of control)

- 控制流是活动图中用于标示控制路径的一种符号。它负责当一个动作或活动节点执行完毕后，将执行主体从当前已完毕的节点转移到过程的下一个动作或动作节点。
- 控制流从活动图的开始标记开始运行，经过顺序、分支等结构引导着各个动作的连续执行。



2. activity(活动)

◆活动代表工作流中的一个任务的执行。活动可以分解成更多的动作。

说明：

◆Action 和activity的区别：

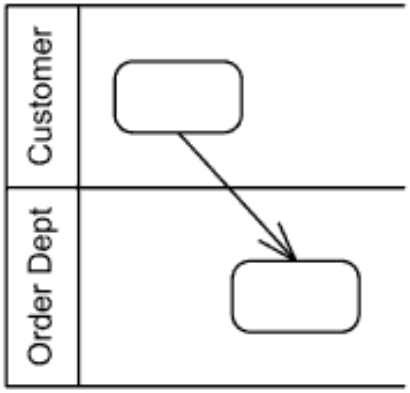
■Action是原子的，而activity可进一步分解。

3. Swimlane(泳道)

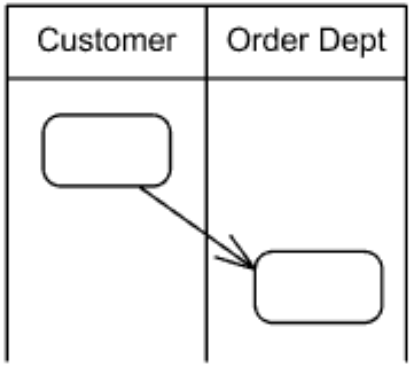
◆由组织内的某个人或者某些人执行的活动图.

说明：

◆每个泳道代表一个责任区。



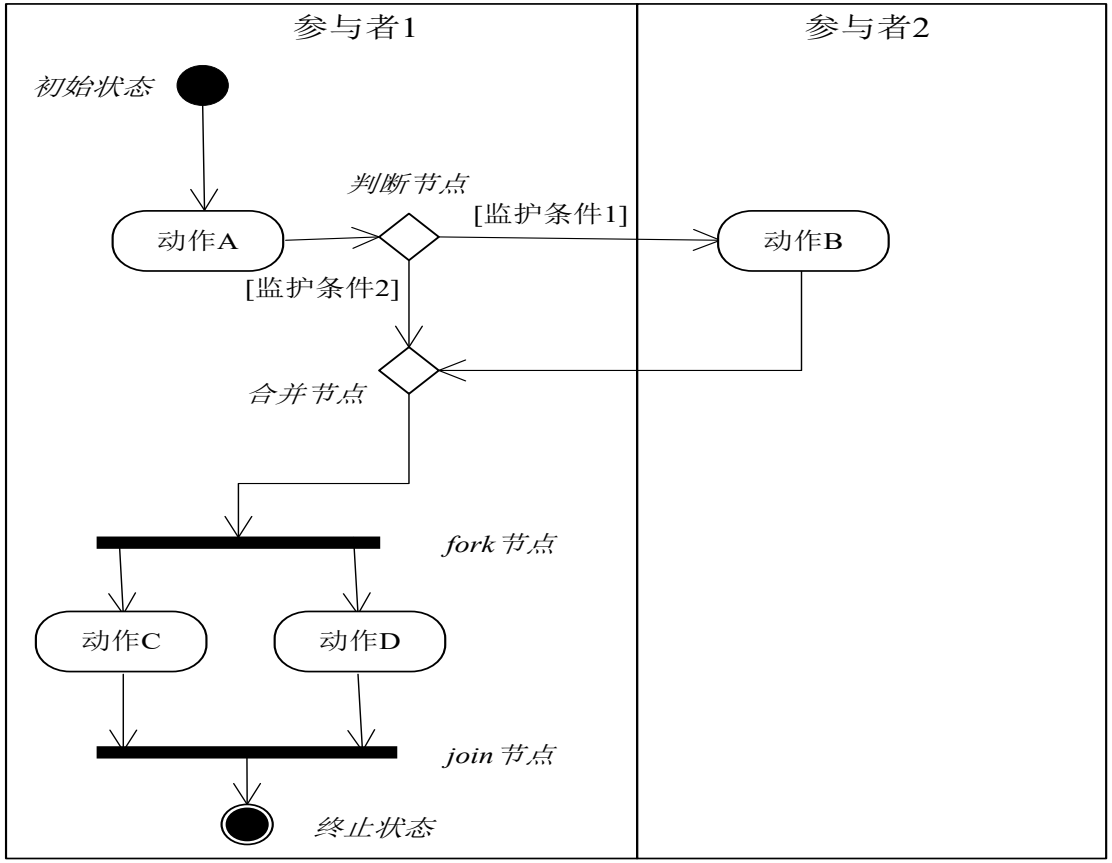
Activity partitions Customer and Order Dept as horizontal swimlanes.



Activity partitions Customer and Order Dept as vertical swimlanes.

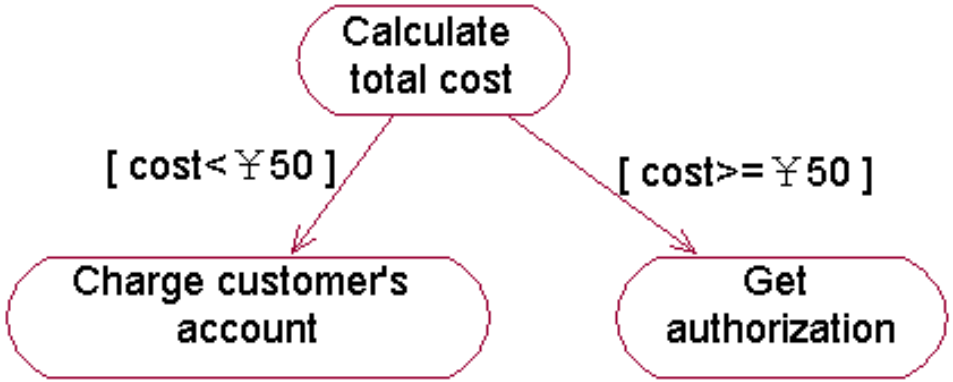
3. Swimlane(泳道)

- 下图是泳道UML建模方式的表述：每个泳道代表一个组织，泳道内部的活动由当前组织来完成。例如，参与者1和参与者2分别代表2个不同的组织。参与者1需要完成动作A、动作C和动作D；参与者2只需要完成动作B。

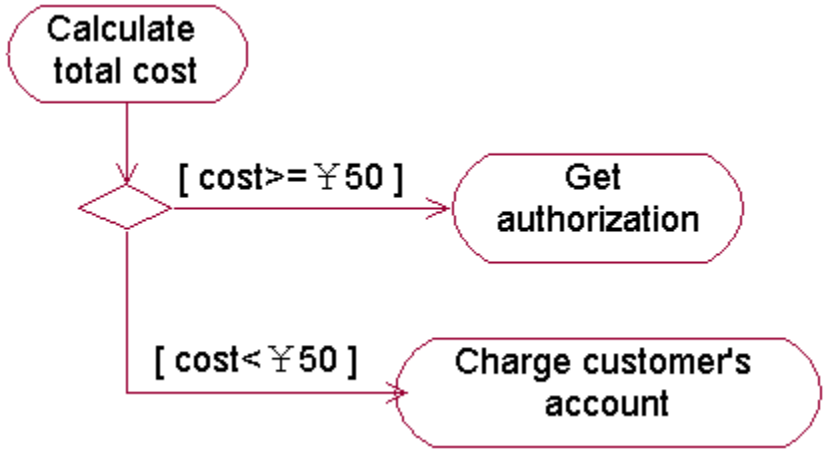


4. 分支与合并(decision node,merge node)

- 如果某个活动有一个以上的后继，那么就需要分别标记每个后继，这种标记方式成为分支。分支是状态图中的一个元素，其中单个触发器导致多个可能的结果，每个结果都有自己的监护条件，即一个活动结束，所有后继都会有测试条件，如果满足某个条件，控制流程就会指向下一个后继的活动。表示分支的两种方法：
- 左面的表示方法类似于状态图中的迁移。



方法1

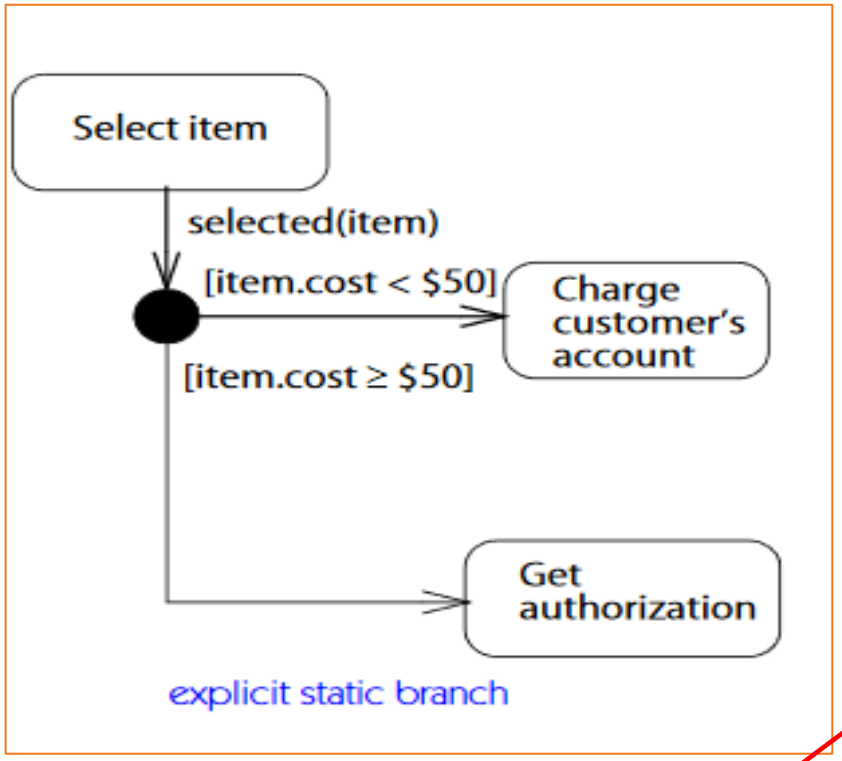


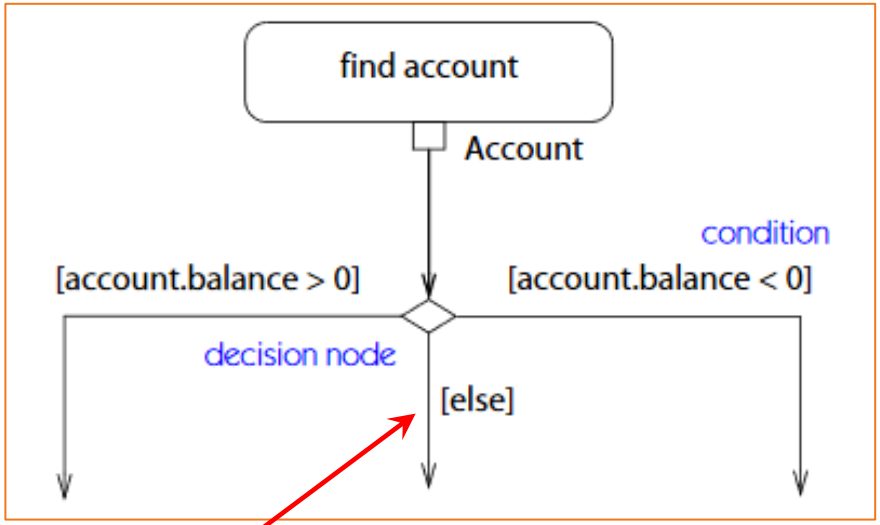
方法2

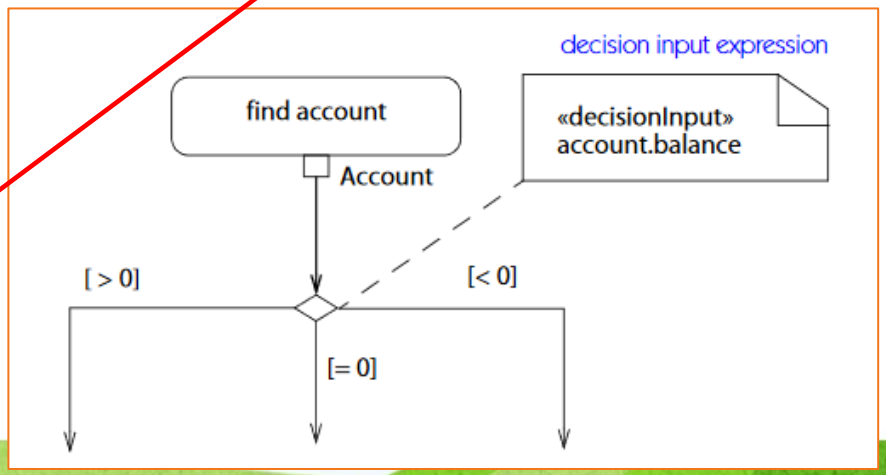
4. 分支与合并(decision node,merge node)

◆第一种分支也可以用一个小黑点(junction node)连接分支。这是来自状态图的方法。

◆可以用一个«decisionInput»把判断条件或者行为放在注释中。





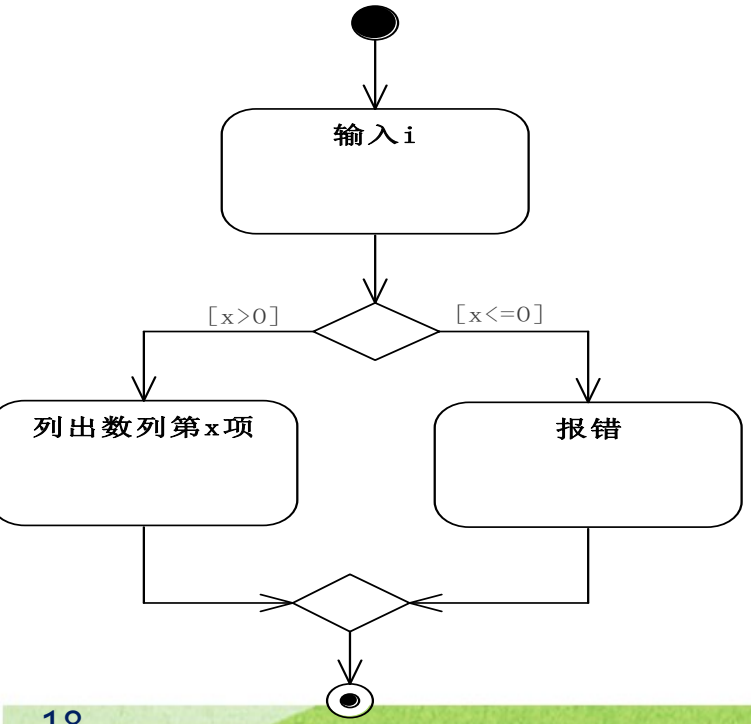


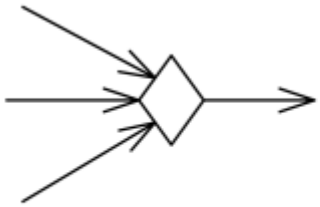
条件可以写else

4. 分支与合并(decision node,merge node)

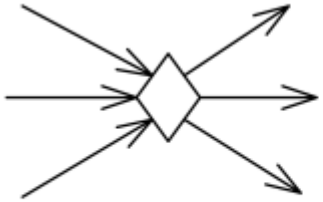
◆合并，统一导出到同一个离开控制流。合并节点仅有逻辑意义而没有时间和数据上的意义：几个动作都指向同一个合并节点也并不意味着这些动作要在进入之后互相等待或进行同步数据之类的操作。合并可以有多个输入分支。

◆分支和合并节点可以合并到一个节点。如右下图。





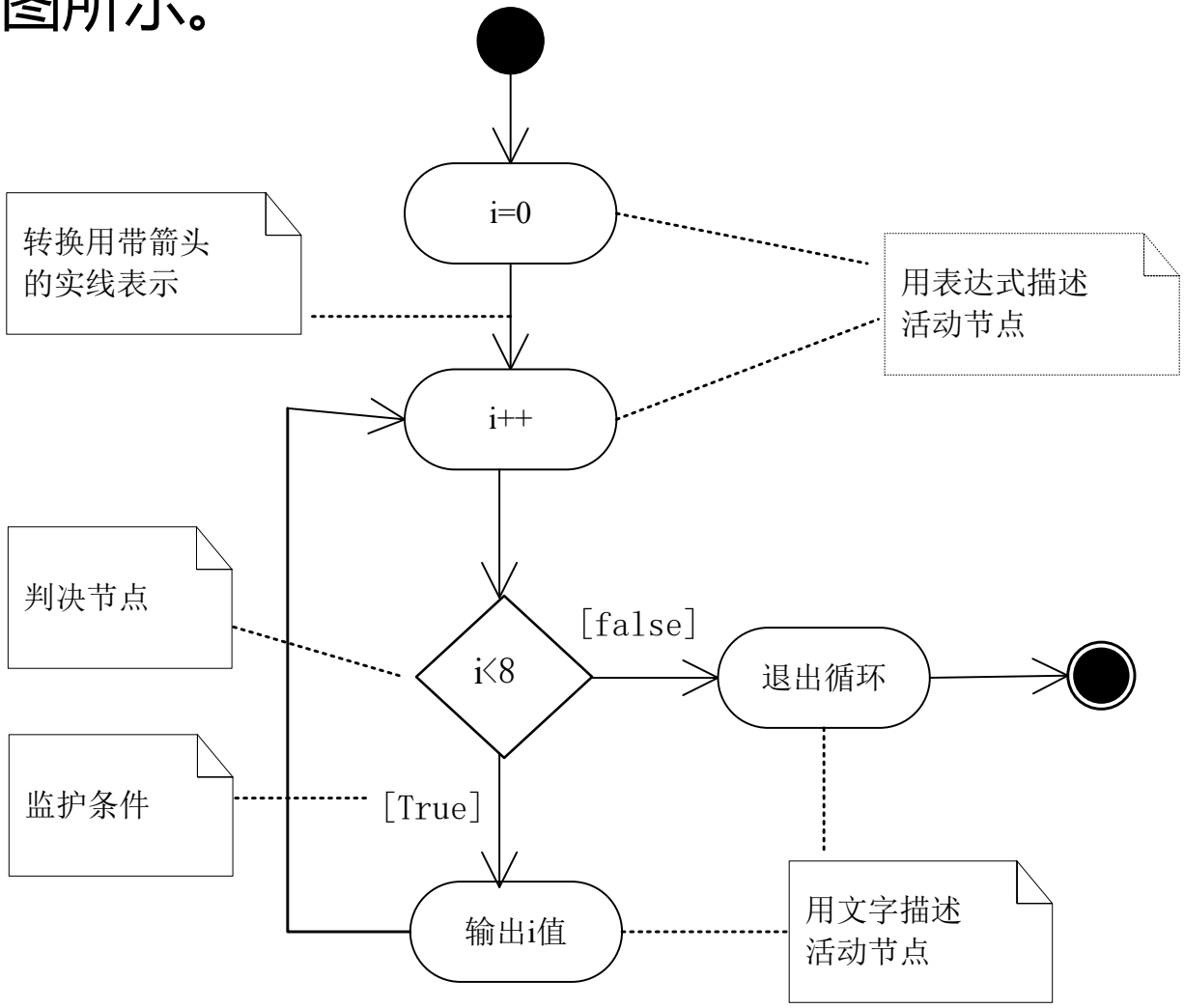
***Merge node** with three incoming edges and a single outgoing edge*



***Merge node** and **decision node** combined using the same symbol*

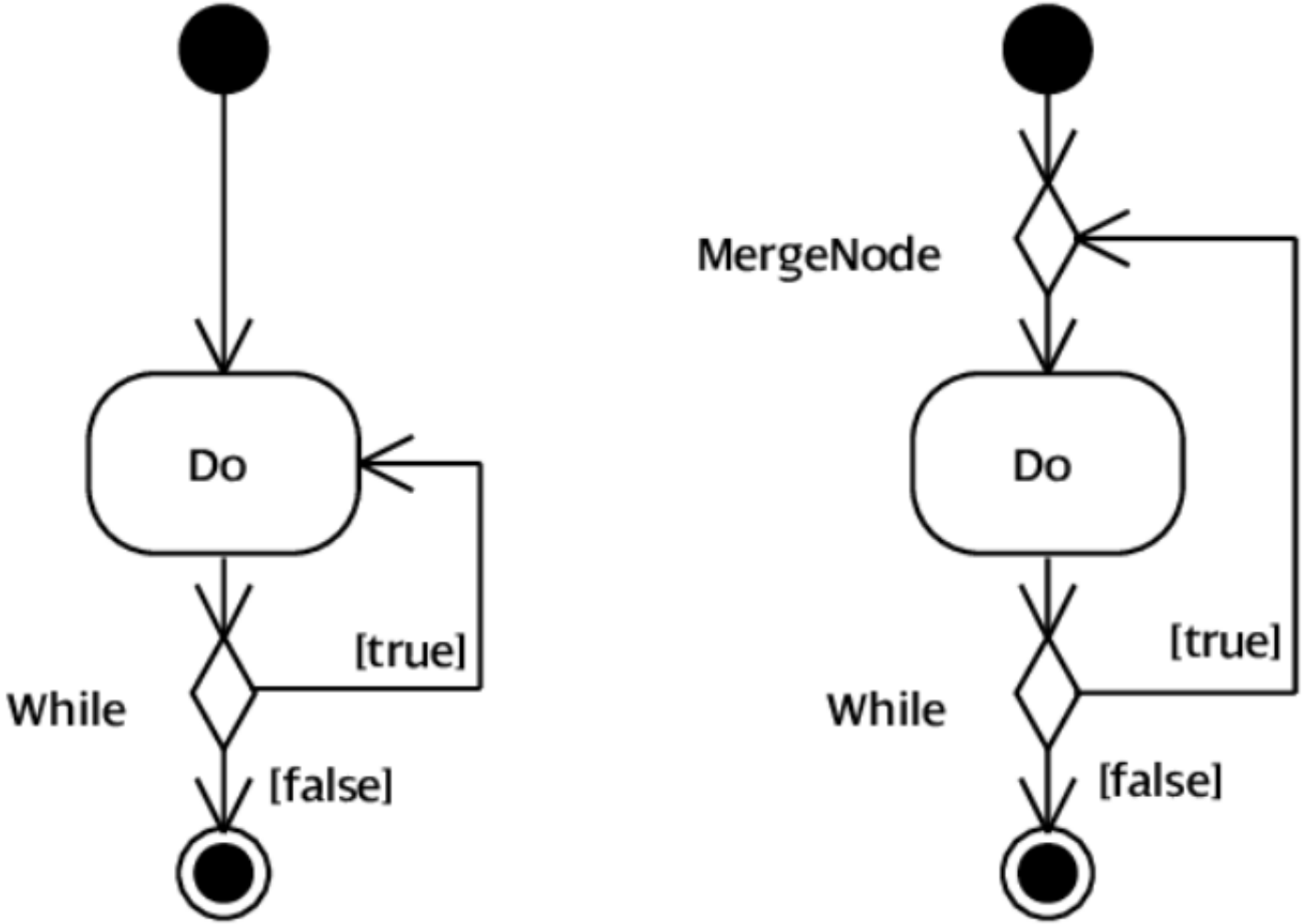
4. 分支与合并(decision node,merge node)

◆用活动图来表示for (i=0;i<8;i++)循环输出0~7八个数字时，如图所示。



用文字和表达式描述活动节点

Do while loop



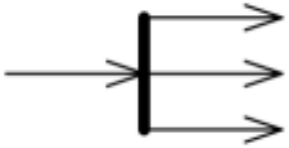
Two realizations of a do-while loop as an activity diagram. Left: UML 1.x, Right: UML 2.x.

5. fork和join(分叉和汇合)

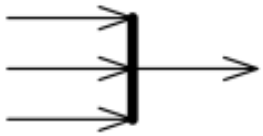
◆UML建模模型中，并发活动分为fork分支和join分支。分叉用来表示将一个控制流分成两个或者多个并发运行的分支，结合用来表示并行分支在此得到同步。当且仅当所有的并发分支（活动）都到达结合点后，活动流程才能进入下一个活动节点。

◆fork是一种复杂的转换，其中一个源状态被两个或多个目标状态替换，从而导致活动状态的数量增加。

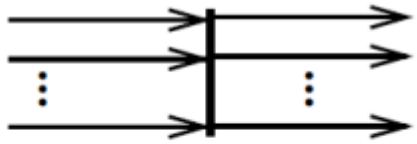
◆join是具有两个或多个源状态和一个目标状态的转换。



fork: 1对3



join:3对1

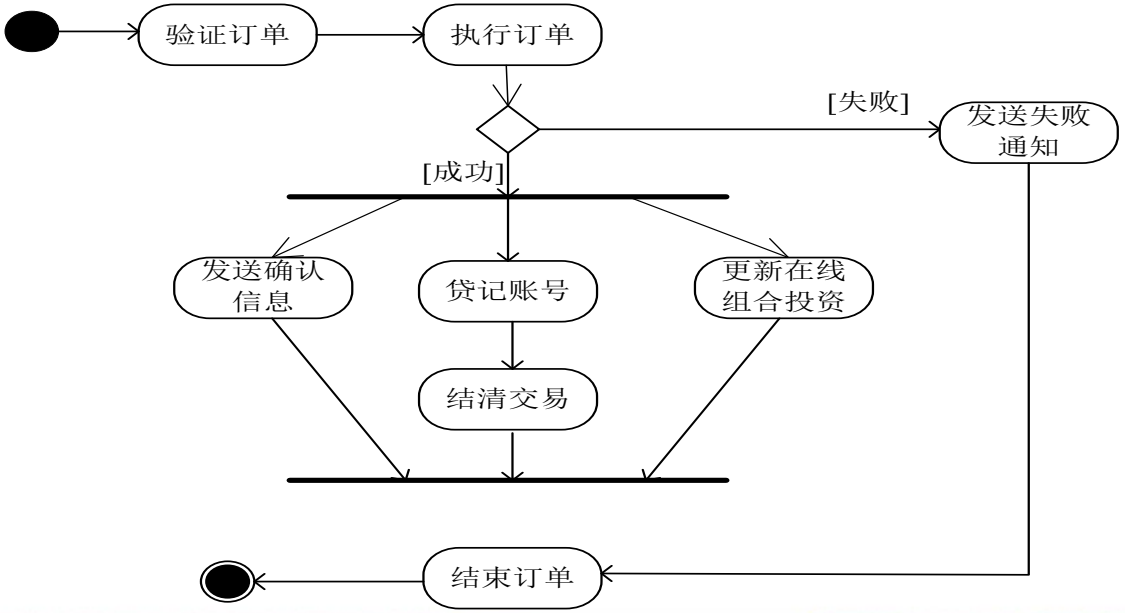


Join node and fork node used together, sharing the same symbol

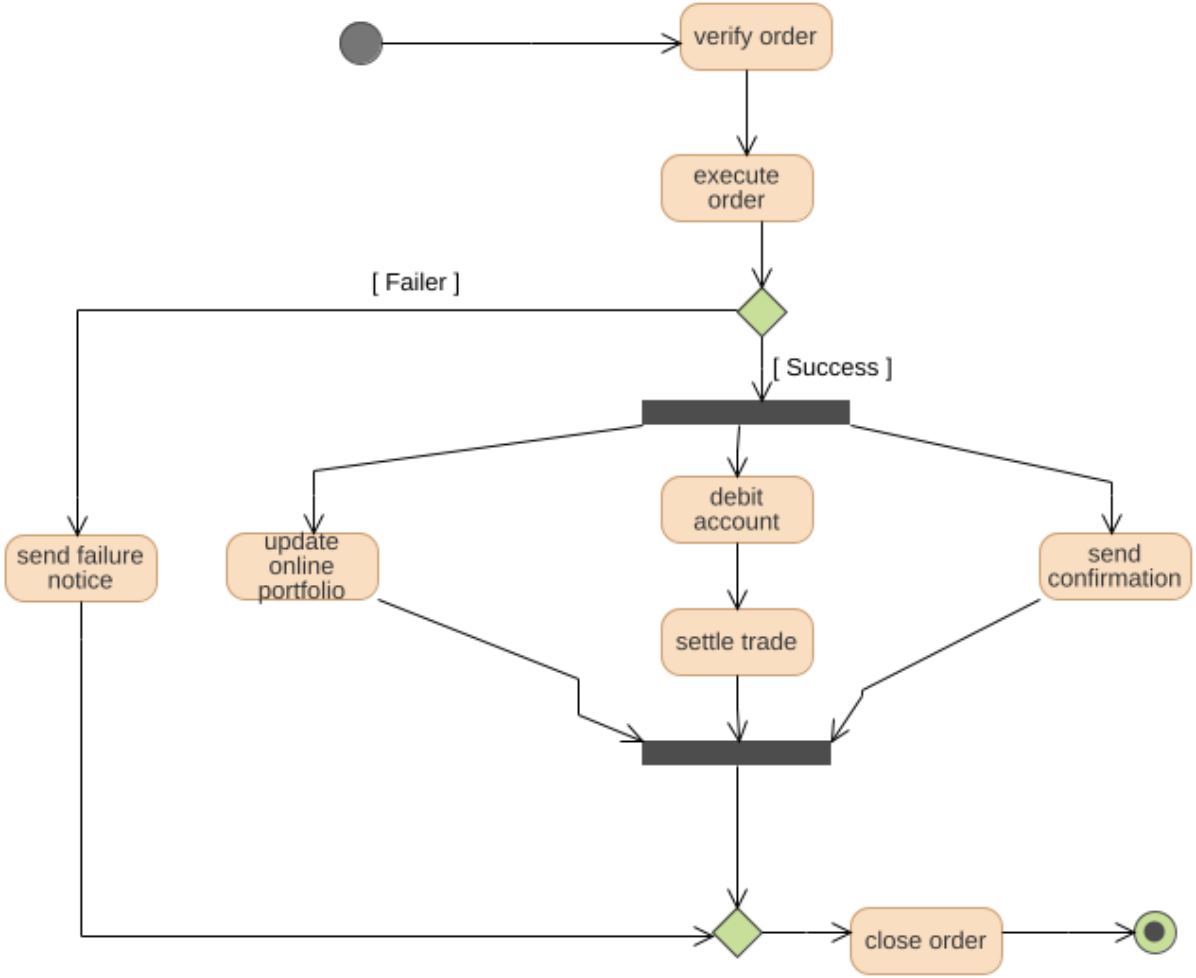
先同步，再分叉

5. fork和join(分叉和汇合)

◆下图显示了处理交易交易订单过程的活动图。系统首先验证客户账户，然后在证券交易所执行订单。如果订单执行成功，系统会产生一个fork分支，它包含3个后继活动：给顾客发送交易确认信息、更新在线组合投资信息以便反映交易结果，以及通过借记账户和迁移现金来与另外一方结清交易。当3个活动完成时，系统会产生一个join分支，将控制合并为单个线程，并结束订单。如果订单执行失败，系统会给客户发送失败通知，并结束订单。

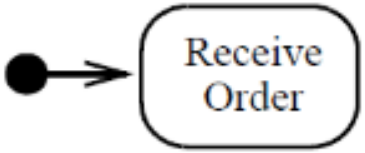


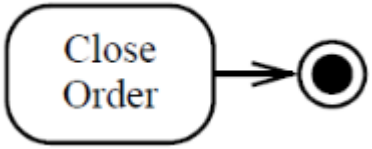
◆对应decision node, 追加merge node:



6. 初始和终止

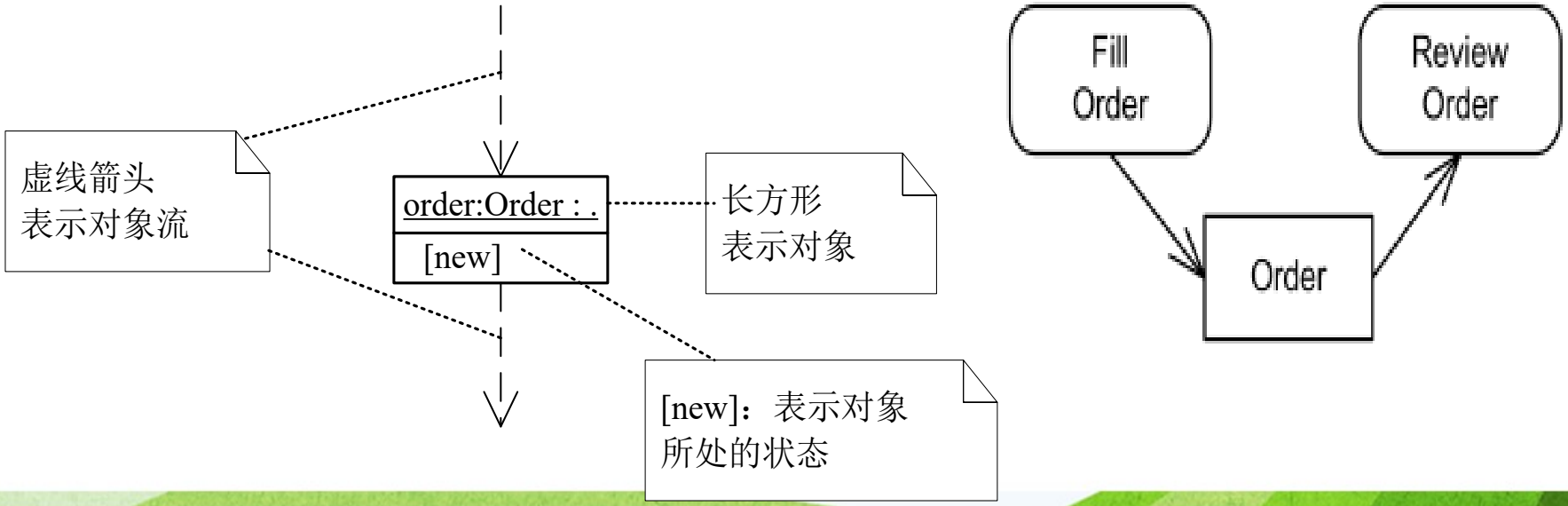
- 实心圆代表活动的开始，实心圆外加空心圆代表活动的结束。
- 当活动图激活时，控制从实心圆开始，并通过输出箭头延伸到最开始的几项活动。
- 当活动达到终止时，全部活动都要完成，活动图的执行结束。





◆活动图中交互的简单元素是活动和对象，控制流（control flow）就是对活动和对象之间关系的描述。详细的说控制流表示动作与其参与者和后继动作之间以及动作和其输入和输出对象之间的关系。而对象流就是一种特殊的控制流。

◆对象流（object flow）是将对象流状态作为输入或输出的控制流。在活动图中，对象流描述了动作状态或者活动状态与对象之间的关系，表示了动作使用对象以及动作对对象的影响。(UML 2标准是实线)



11.1.3活动图的绘制

①整理出场人物，抽取出角色

- ◆角色是现在的业务中出现的人、组织、系统等。
- ◆角色对应着活动图中的活动分区(Activity Partition)。

②整理角色间交换的信息

- ◆这些信息对应着活动图里的对象节点(Object Node)。

③找出开始节点和终止节点

- ◆一个活动图只有一个开始节点。
- ◆如果存在多个分支，可能会有多个终止节点。

④找出各个角色的活动

- ◆分析业务描述中的动词，获得各个角色的活动。

⑤做成活动图

- ◆信息、开始/终止节点、活动放置到合适的地方，并连接起来。

11.1.3活动图的绘制-例子

根据以下的图书馆借书的业务说明，画出借书的活动图。

读者来图书馆借书，可能先查询书库的图书记录。查询可以按书名、作者、图书编号、关键字查询。查询有3种结果，如果查到则记下书号，交给工作人员，然后等候办理借书手续。如果该书已经被全部借出，则可做借书登记，等待有书时被通知。如果图书馆没有该书的记录，则做缺书登记。

办理借书手续时先要出示图书证，没有图书证则去申请图书证。如果借书数量超出规定，则提示“借书数量超限，不能继续借阅”。工作人员登记借阅人信息、借阅的图书信息、借出时间和应还书时间。系统自动修改书库的图书记录、读者库信息。

11.1.3活动图的绘制-例子

步骤1：整理出场人物，抽取出角色

读者

工作人员

系统

步骤2：整理角色间交换的信息

书名、作者、图书编号、关键字

书号

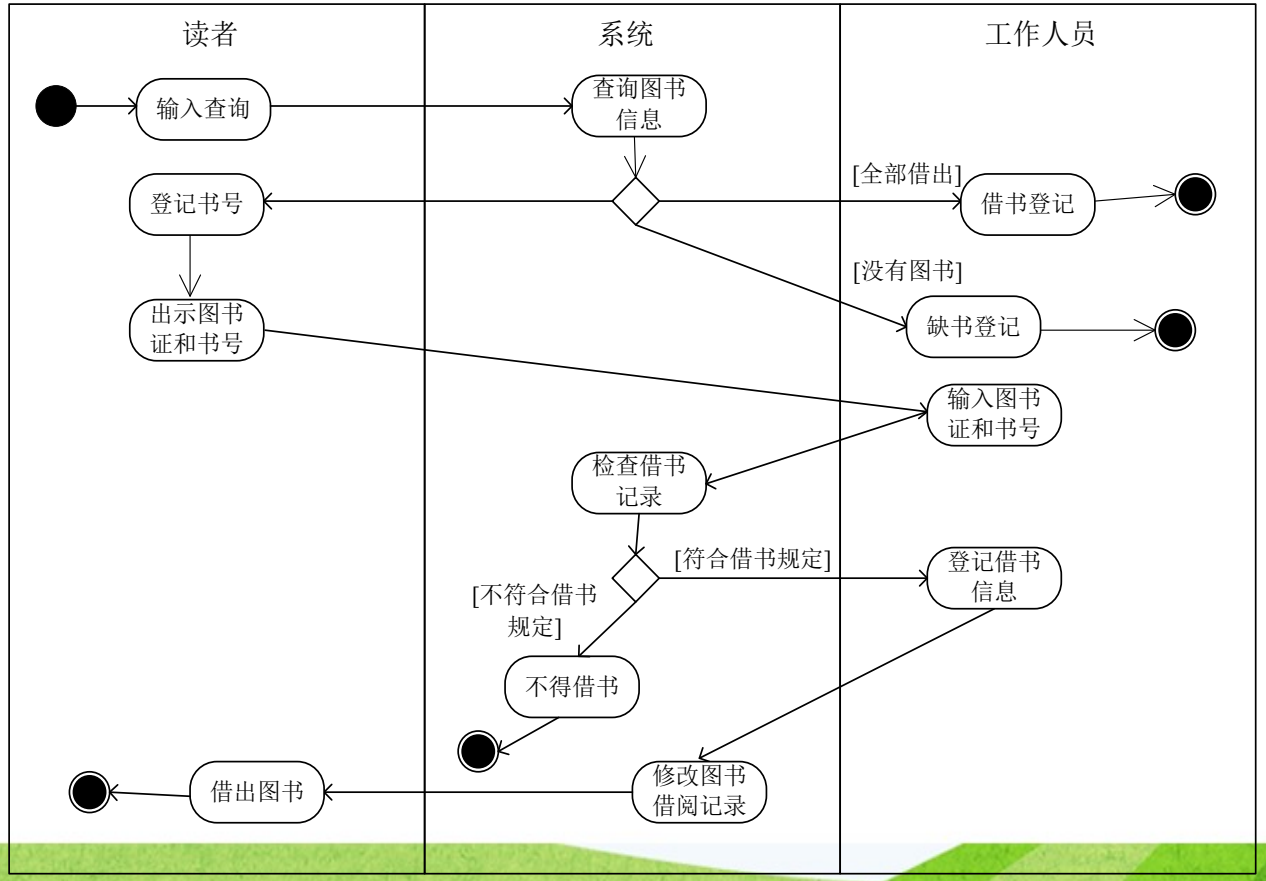
图书证

步骤3：绘制活动图

如下图所示。

11.1.3活动图的绘制-例子

在借书活动图中，包含3个泳道，分别是读者、系统和工作人员。借书活动开始于读者，结束活动有多个：借书登记活动完成后就结束、缺书登记完成后结束、不得借书完成后结束以及借出图书完成后结束。查询图书信息活动结束后，开始根据查询结果进行分支处理，分为没有图书、全部借出和有可借出的图书。另外，系统检查图书记录有两个分支，一个符合规定，一个不符合规定，分别都包含有特定的后继活动。



11.1.4活动图与状态图比较

1. 活动图和状态图描述的重点不同：
   - ■活动图描述的是从activity到activity的控制流，而状态图描述的是对象的状态及状态之间的转移。
2. 活动图和状态图使用的场合不同：
   - ■对于以下几种情况可以使用活动图：
     - □分析用例
     - **□理解涉及多个用例的工作流**
     - □处理多线程应用
   - ■对于下面的情况要使用状态图：
     - □显示一个对象在其生命周期内的行为。

说明：如果要显示多个对象之间的交互情况，可用顺序图或协作图。

活动图和顺序图区别

- https://www.geeksforgeeks.org/difference-between-sequence-diagram-and-activity-diagram/

Differences Between Sequence Diagram and Activity Diagram

Sequence Diagram	Activity Diagram
The Sequence diagram represents the UML, which is used to visualize the sequence of calls in a system that is used to perform a specific functionality.	The Activity diagram represents the UML, which is used to model the workflow of a system.
The Sequence diagram shows the message flow from one object to another object.	The Activity diagram shows the message flow from one activity to another.
Sequence diagram is used for the purpose of dynamic modelling.	Activity diagram is used for the purpose of functional modelling.
Sequence diagram is used to describe the behavior of several objects in a single use case	Activity diagrams is used to describe the general sequence of actions for several objects and use cases.
Sequence diagram is mainly used to represent the time order of a process.	Activity diagram is used to represent the execution of the process.

活动图建模风格

- 建模风格1：确保从决策点出来的每个转移都有一个警戒条件。
  - 这可以确保建模人员已经考虑到这个决策点的所有可能情况。
- 建模风格2：确保决策点上的警戒条件形成一个完备集。
  - 不管什么情况，要一定能够可以从决策点离开。
  - 例如，像x < 0和x > 0这样的警戒条件是不完备的，因为当x等于0时会发生什么并不清楚。
- 建模风格3：警戒条件不要重叠。

◆建模风格4：确保每个分叉只有一个进入转移。

■当发现需要几个转移到同一个分叉时，表明建模人员或者遗漏了一个活动，或者在该点事实上并不需要并发活动。

◆建模风格5：确保每个汇合只有一个退出转移。

■当某个汇合点需要多个退出转移时，表明仍然需要并发的活动，因此，把汇合点沿着总体活动过程向前移动。

◆建模风格6：要小于5条泳道。

- ■泳道的一个缺点是使得建模人员无法自由地在图中排列活动以节省空间，因此会增加图的大小。泳道数目越多，这个问题越严重。

更多活动图举例

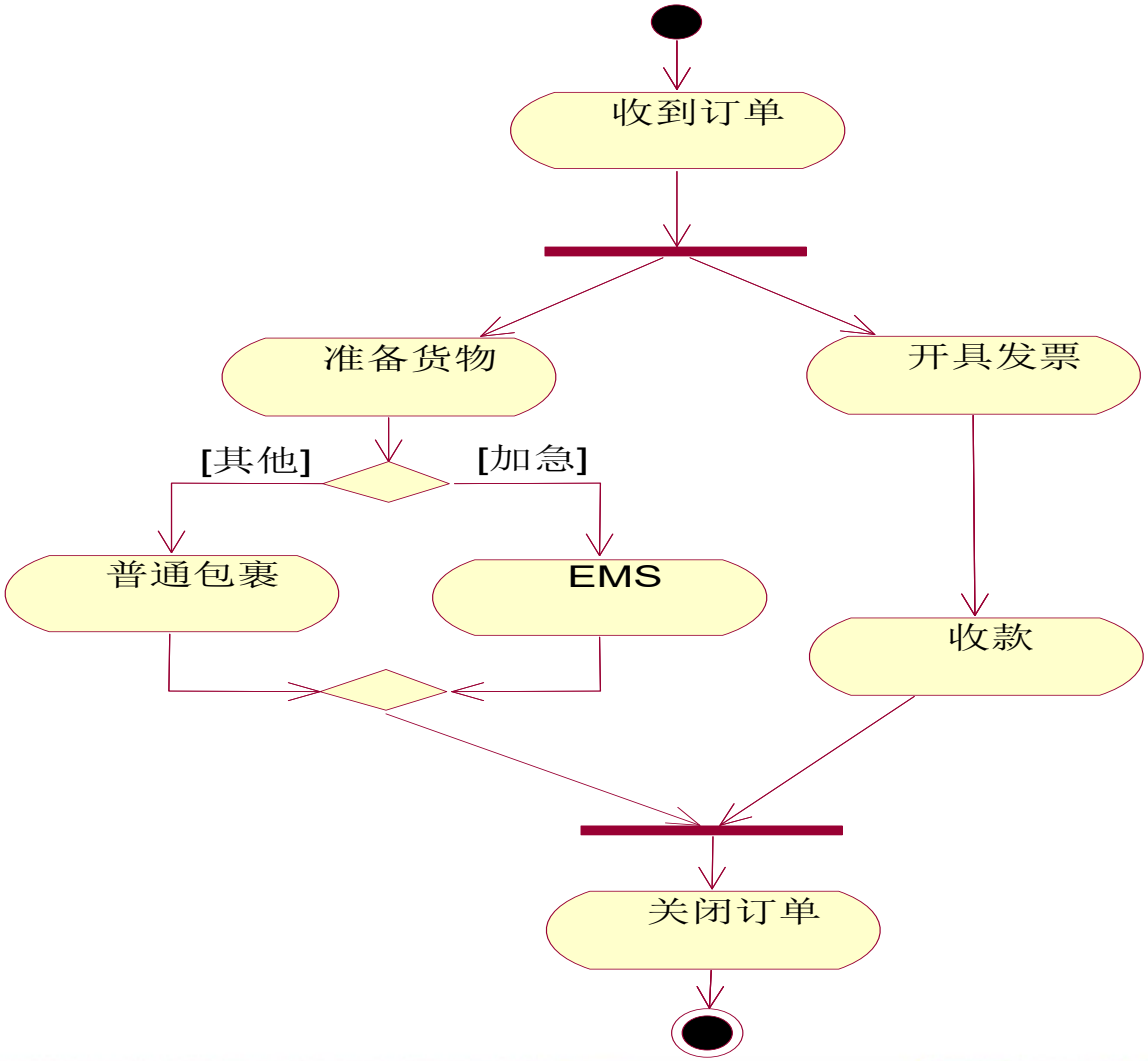
- 按照活动图表示的信息不同，可将活动图分为
  - 简单活动图、
  - 标识泳道的活动图、
  - 标识对象流的活动图和
  - 复合活动图。

简单活动图

◆某公司销售人员收到订单后，一方面通知仓库管理员准备货物，另一方面通知财务人员开具发票和收款。仓库管理员准备货物时，如果是加急货物，就通过EMS运送，否则通过普通包裹运送。当货物、发票和收款都完成后，关闭订单。

销售流程的简单活动图

◆该图既没有标识活动的执行者，也没有标识在活动执行过程中创建了哪些对象。其中，每个活动都是简单的活动。



展示泳道的活动图

◆每个泳道用一条垂直的线将它们分开，并且每个泳道都必须有一个唯一的名称（泳道的名称用泳道执行者来命名）

◆例如本例中的仓库人员、销售人员和财务人员是各自泳道活动的执行者。通过泳道不仅体现了整个活动控制流，还体现出了每个活动的执行者。

展示泳道的活动图

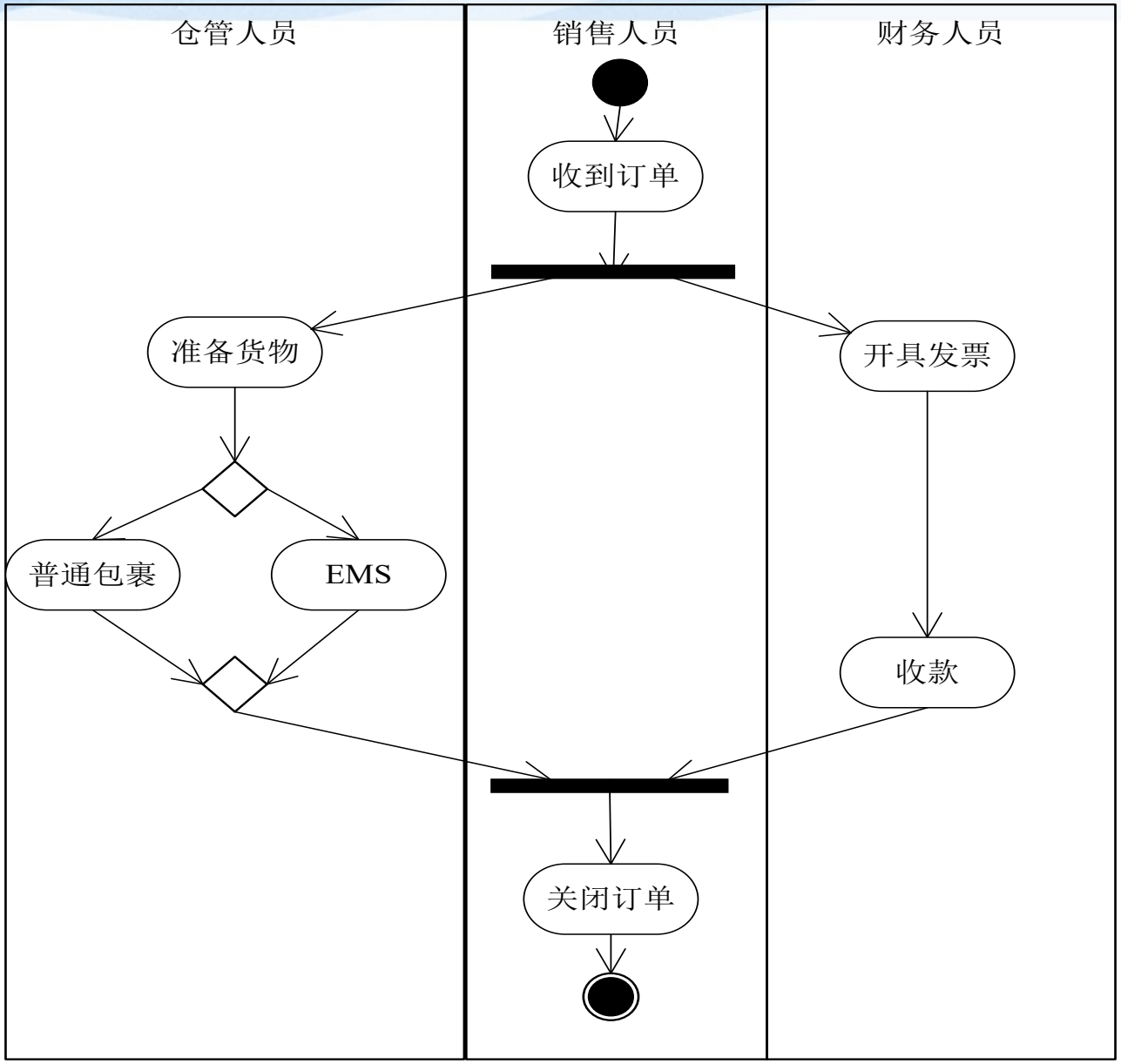
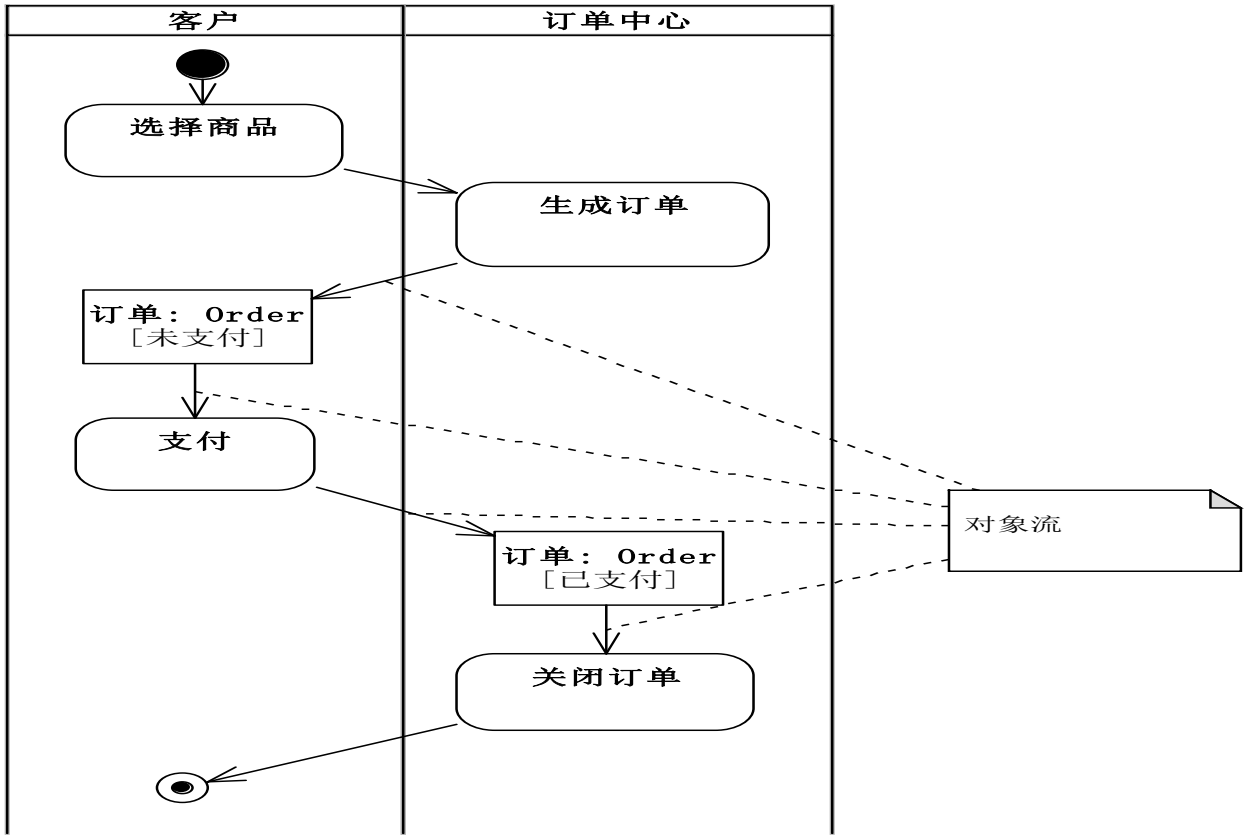


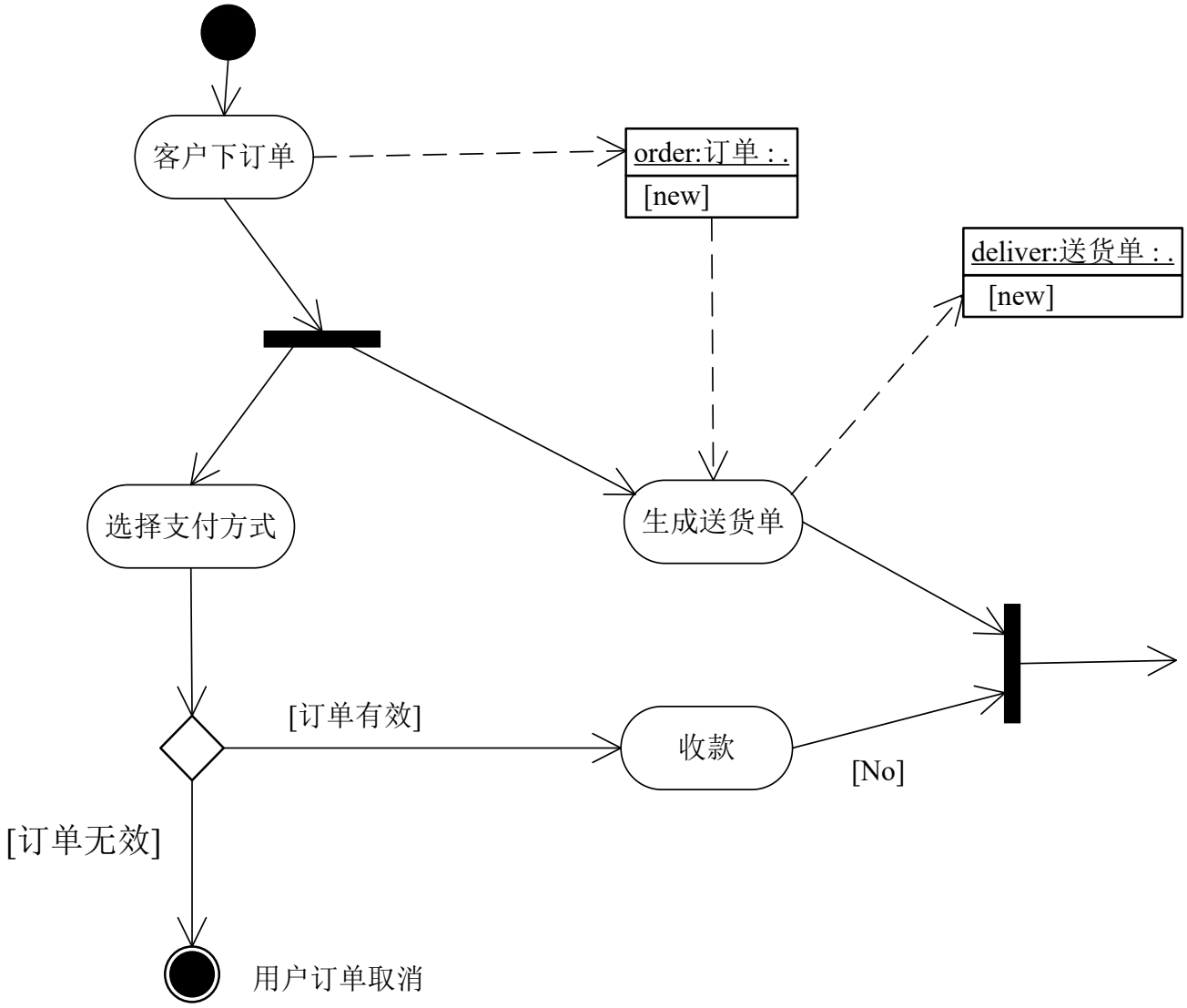
图8-9 展示泳道的活动图

展示对象流的活动图

- 在UML中，用户可以在活动图中标出一个对象的状态和属性值的变化。
- 在活动图中，表示对象的方法如所示。



订单处理”活动的一个片段。



展示参数的活动图(别针，pin)

- 用一个小矩形框表示参数（参数是一个对象节点）。
- 参数有2种，分别是输入参数、输出参数。参数都标识在活动节点的边界上。
- 箭头代表参数的方向，参数名写在别针旁边。

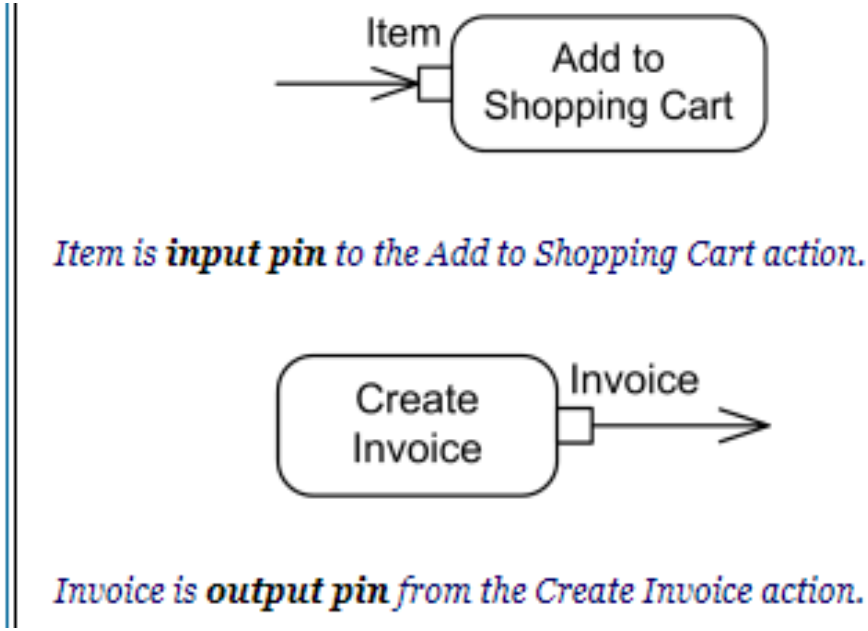
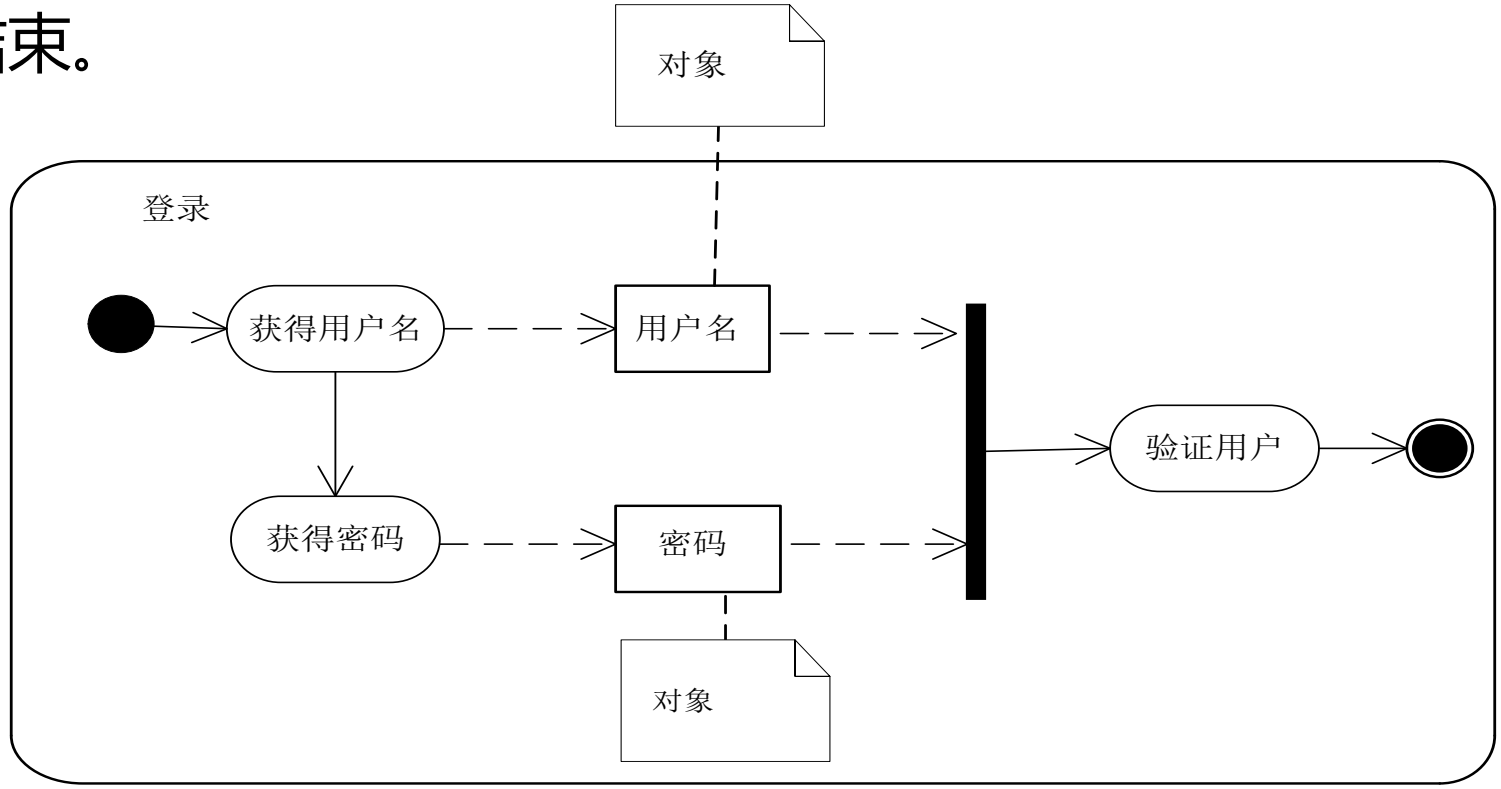


图8-12 标识活动节点的输入参数和输出参数

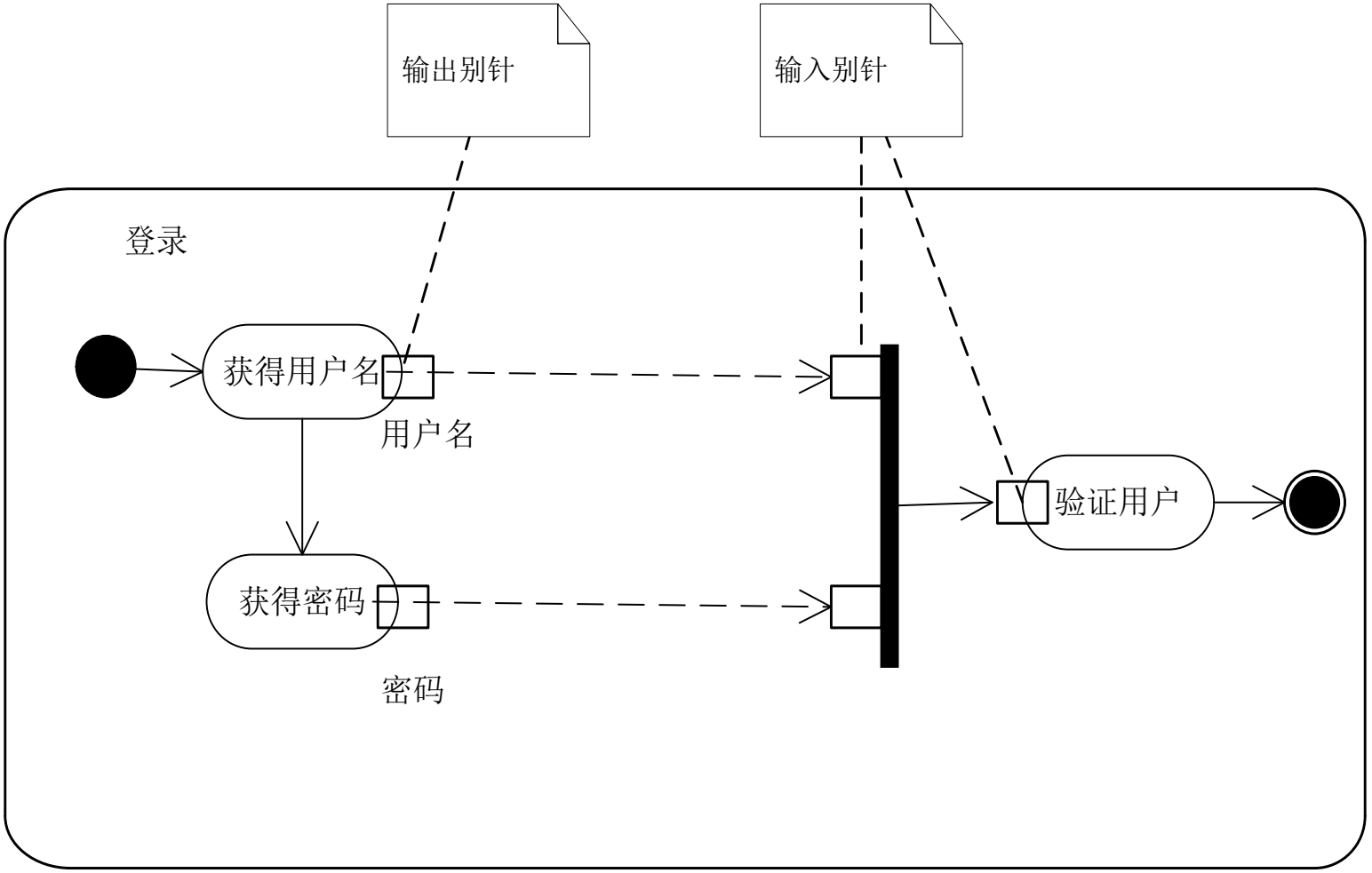
展示别针的活动图

- 下图是一个登录活动图，它展示了对象流。
- 这个活动图展示了2个对象，当执行”获得用户名”时，产生对象”用户名”；当执行”获得密码”时，产生对象”密码”。
- 当这2个对象输入到活动”认证用户”时，用户被认证，活动结束。



展示对象流的活动图

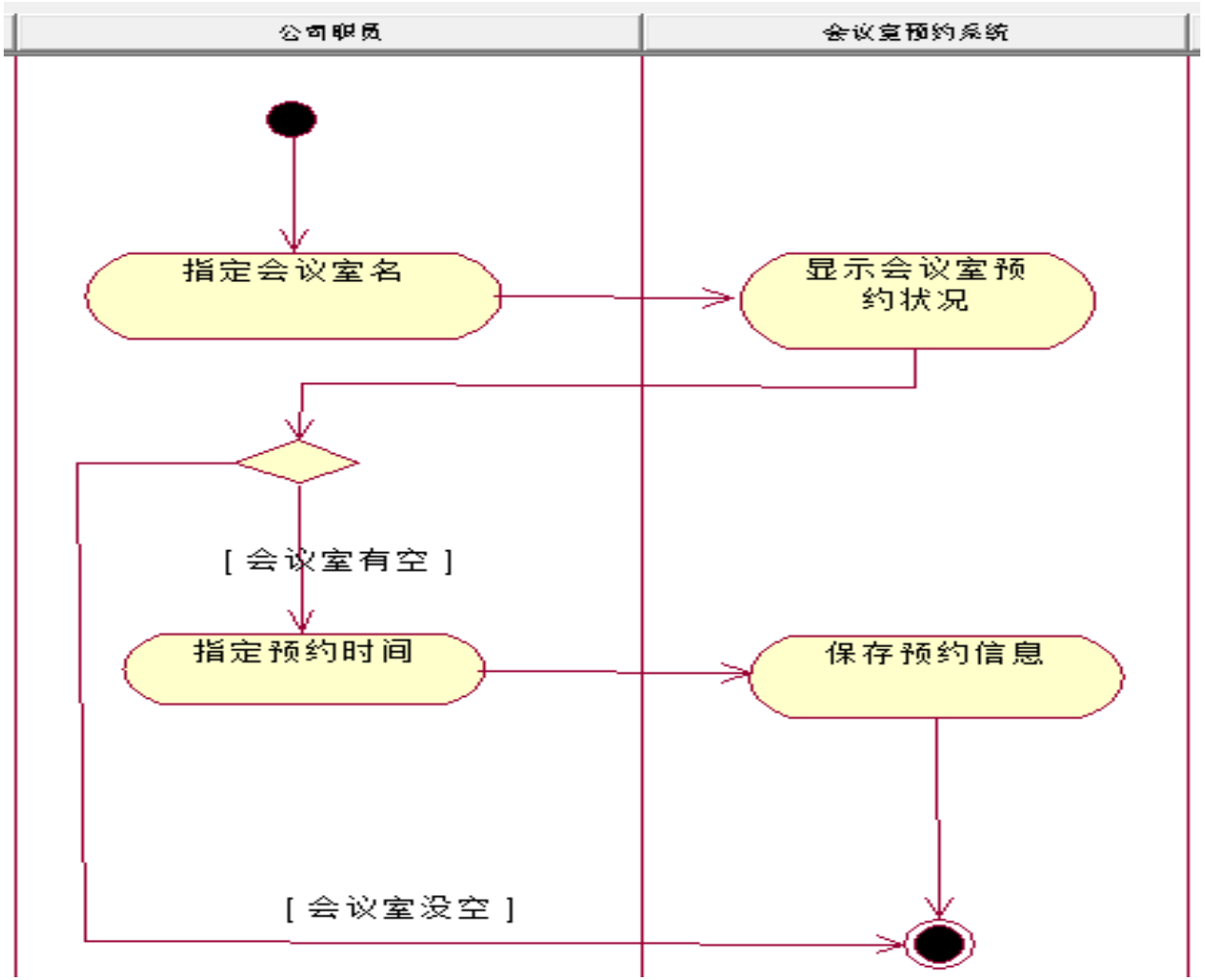
当用别针代替对象后，就绘制出等价的活动图，如图所示。



展示别针的活动图

◆使用会议室预约系统预约会议室时，职员指定会议室名后系统会显示该会议室的预约状况。如果该会议室有空，职员指定预约时间，系统保存预约信息。

◆注意标注警戒条件



11.2包图-概述

- 为了理解模型的可重用性以及可见性，大的模型必须由较小的单元组成。为了有效地组织和管理这些小单元，包的概念被提出。包是拥有和管理模型内容的一般的组织单元。任何元素都可被包所拥有。
- 包是模型的一部分，模型的每一部分必须属于某个包。建模者可以将模型的内容分配到包中。但是为了使其能够工作，分配必须遵循一些合理的原则，如公用规则、紧密耦合的实现和公用观点等。
- UML对如何组包并不强制使用什么规则，但是良好的结构会很大地增强模型的可维护性。

11.2.2包图

- 包图

包图描述的是模型中的包及其包含的元素组合，是维护和控制系统总体结构的重要建模工具。包图是一种将UML中的建模元素组织成组的通用机制。

包的作用：

◆用于组织模型中的元素以便更容易理解；

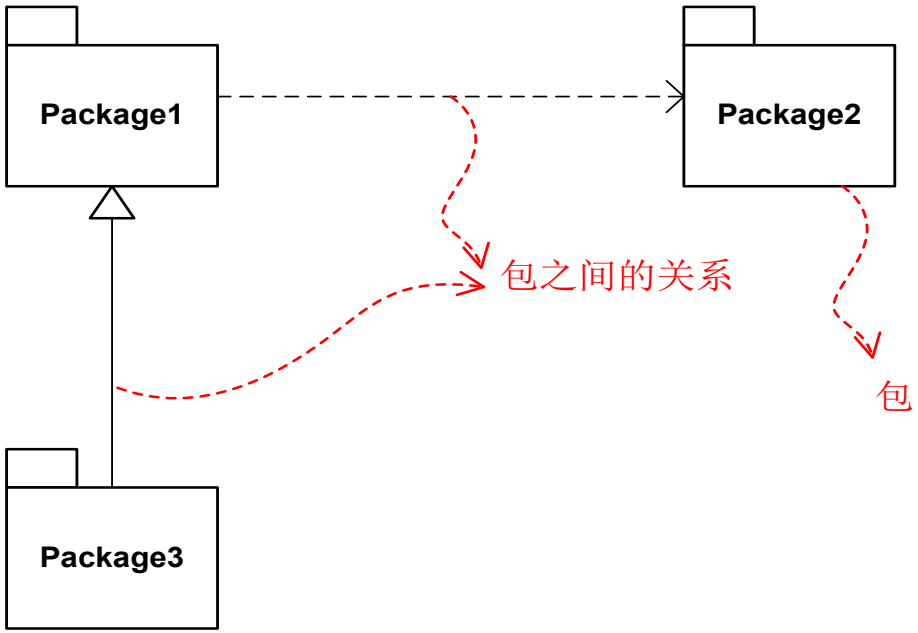
◆对包中元素进行可见性控制。

- 说明：

包中可以包含其它建模元素，如类（class），接口（interface），组件（component），用例（use case），…等。包可以嵌套，但嵌套层次不要过深。包没有实例，即在系统运行时见不到包。

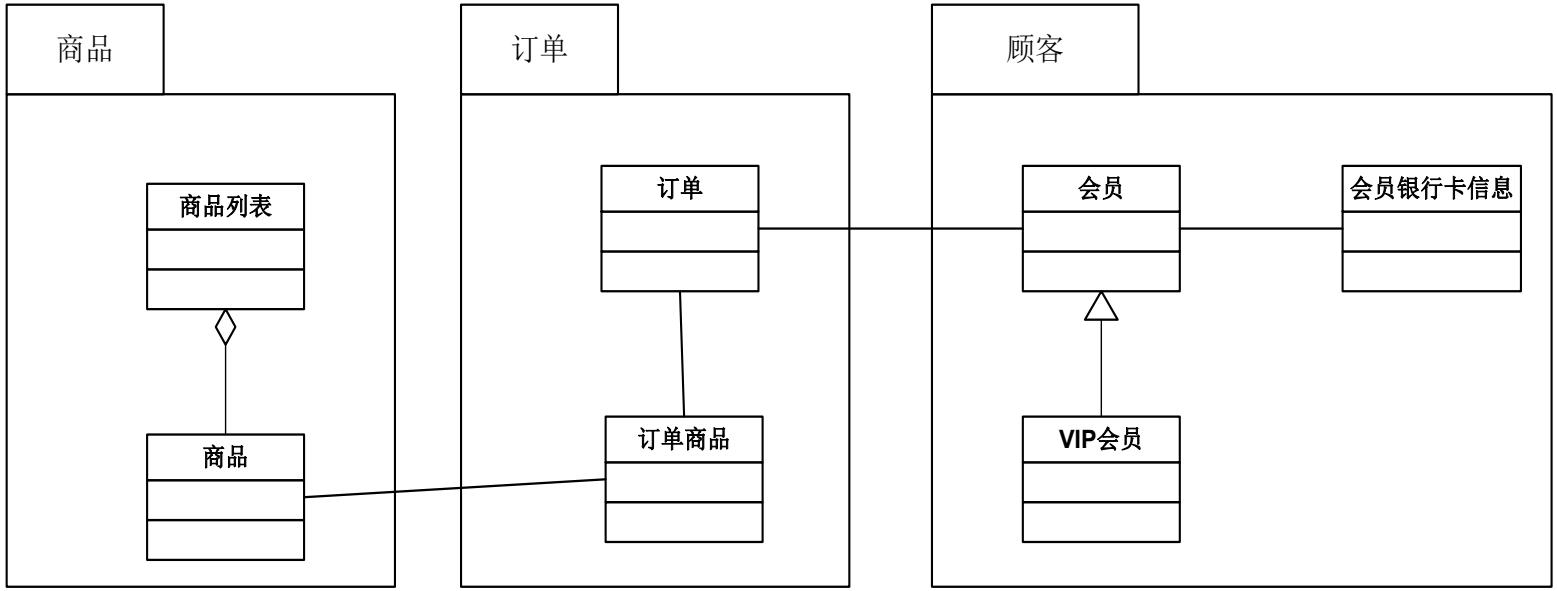
11.2.2包图-概要

- 对UML的模型元素按照意义，功能等进行分组整理的时候使用。
- 通过对UML的模型元素进行分组整理，可以提高模型的可读性和可维护性。
- 包图可适用于UML的所有模型元素。
- 包图内部还可以再含有包。
- 包图的组成元素：
  - 包（package）
  - 包之间的关系（依赖，泛化）



用包对类图进行分组整理的例子

◆ 看一个用包对类图中的类进行分组整理的例子：

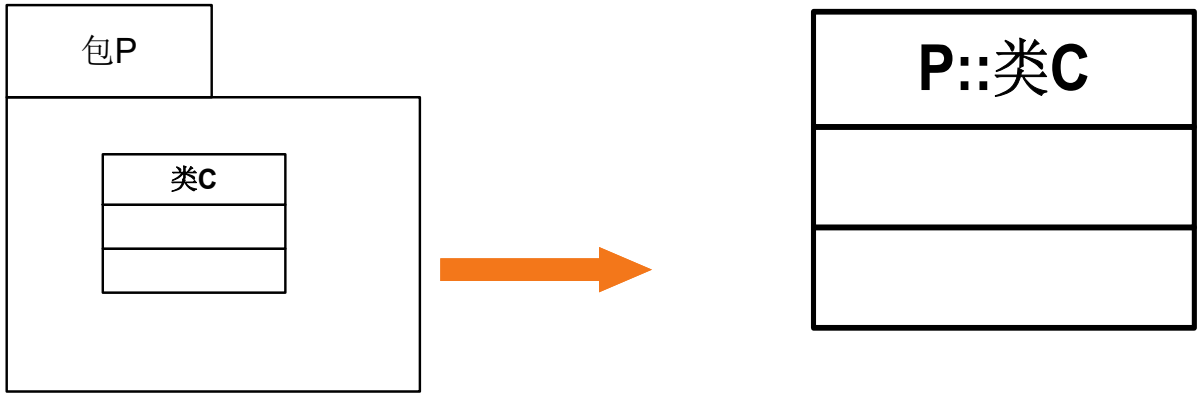


把类图中的类根据其用途划分为三个包—“顾客”包，“订单”包和“商品”包。

用包名来修饰类名

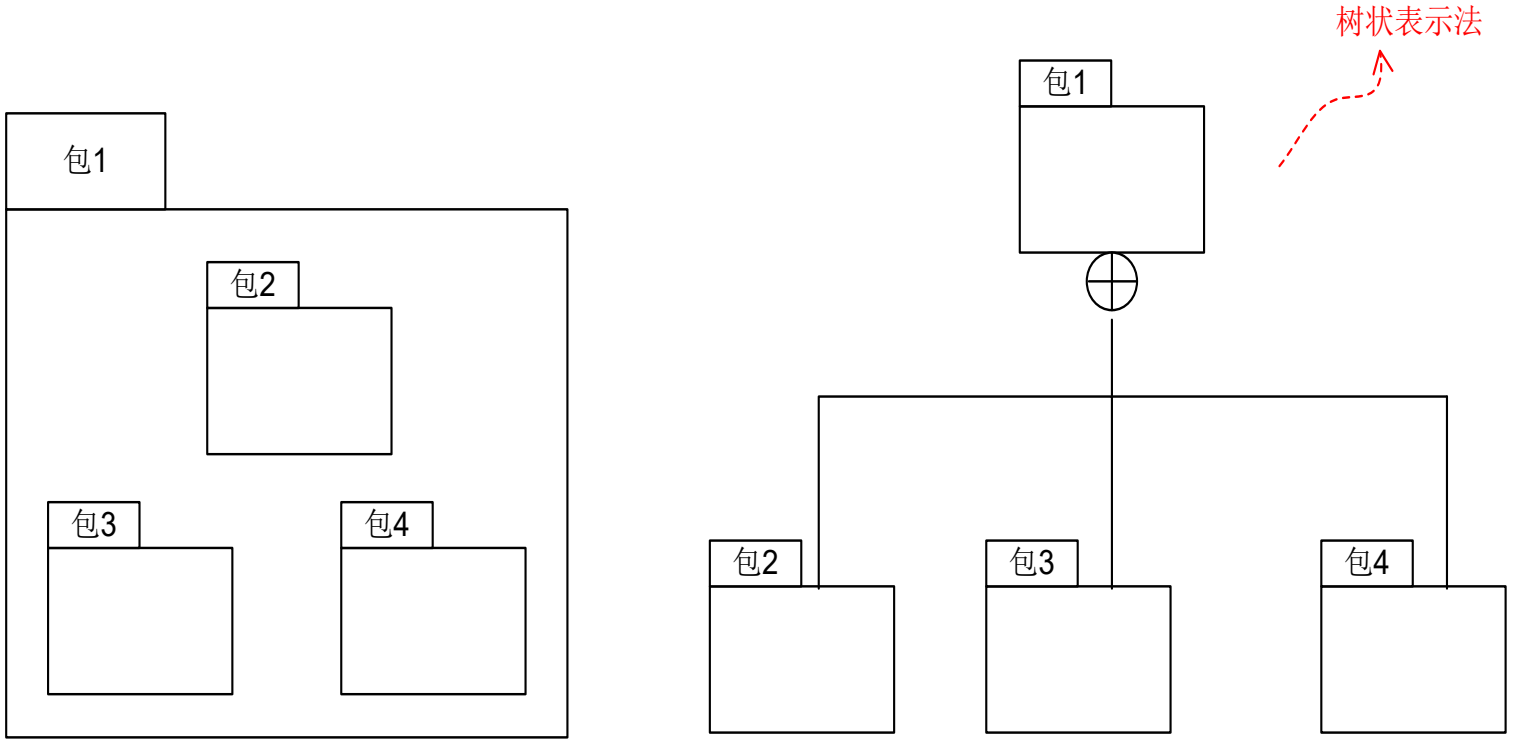
- 当需要明确一个类和其所属的包之间的所属关系时，可用如下方式来对类名进行修饰：

包名：：类名



包的层次化

- 包中还可以含有包，把这种情况称为包的层次化。
- 包的层次化表现有两种方式：

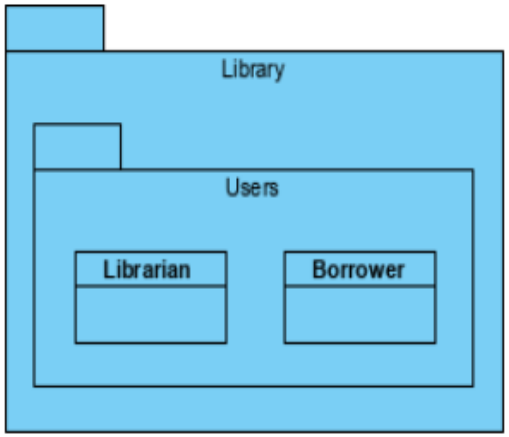


包的层次化

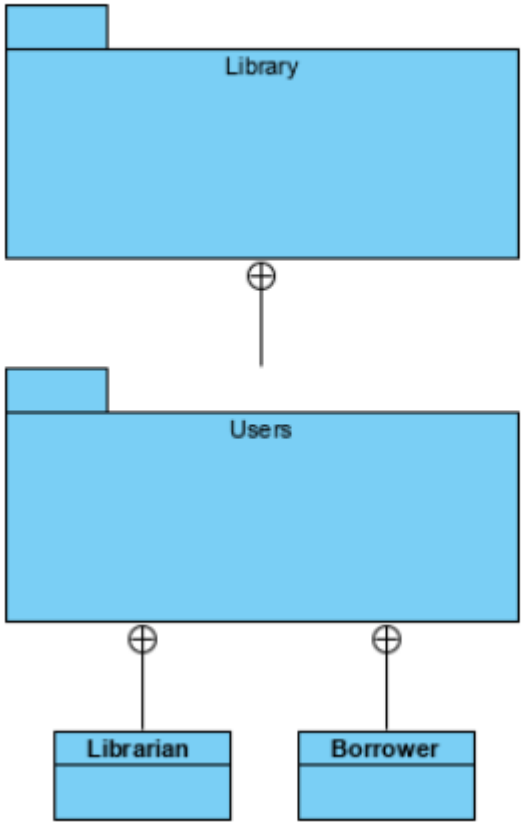
- 包中还可以含有包，把这种情况称为包的层次化。
- 包的层次化表现有两种方式：

- Qualified name of the class Librarian in the figure below is

Library::Users::Librarian

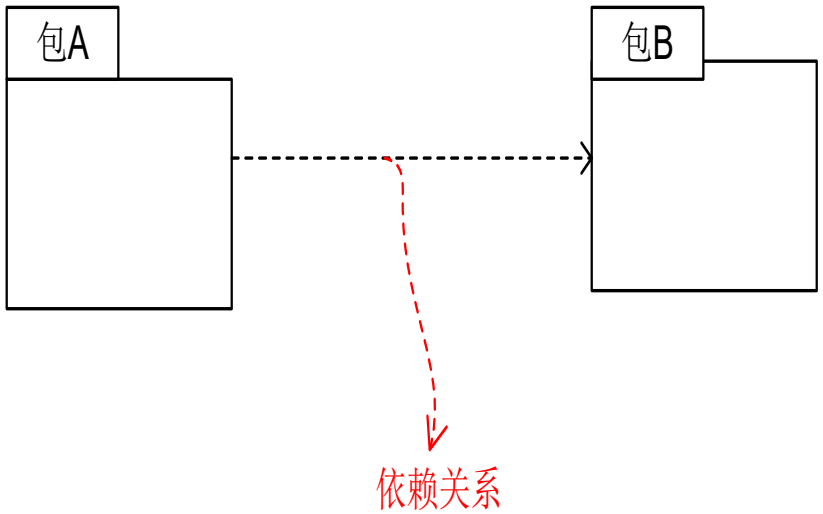


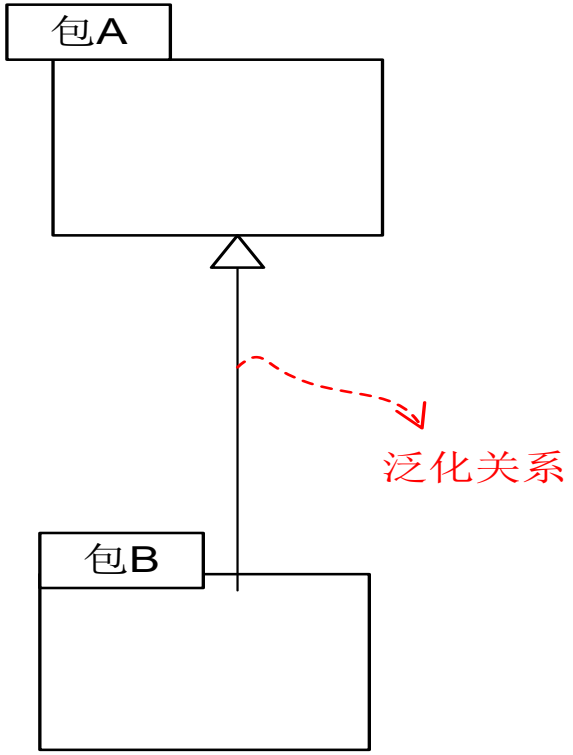
Alternative Representation of Package



包之间的关系

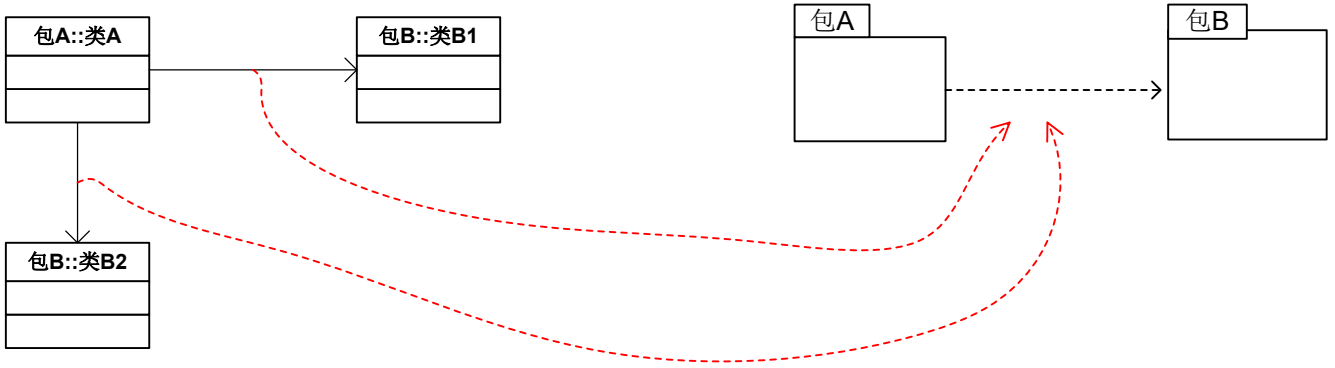
- 包之间可以具有依赖关系或者泛化关系。
- 包之间的关系取决于包内的元素之间的关系。
  - ■两个包内的元素见具有使用上关系时，这两个包之间具有依赖关系。
  - ■两个包内的元素具有泛化关系时，这两个包之间具有泛化关系。



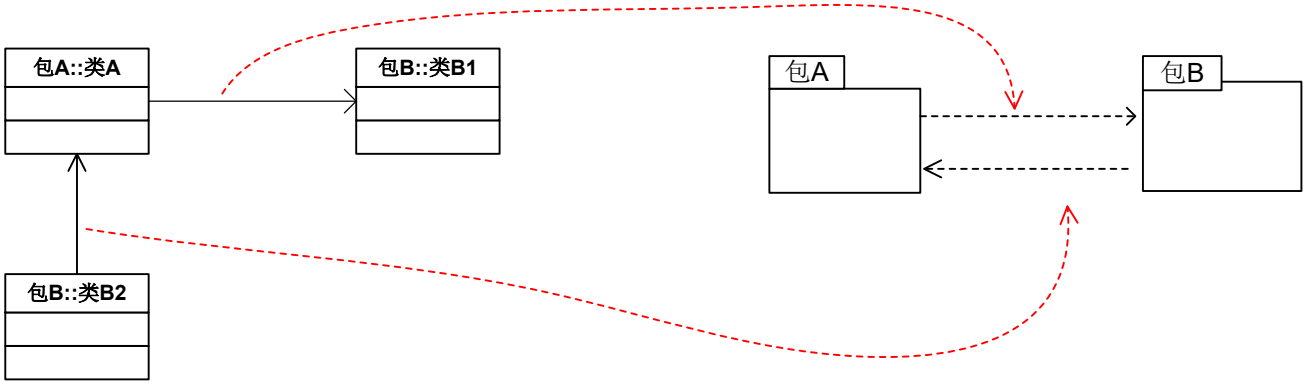


包之间依赖关系（单向）

- 当两个包内的元素只有单向的使用关系时，两个包之间具有单向依赖关系



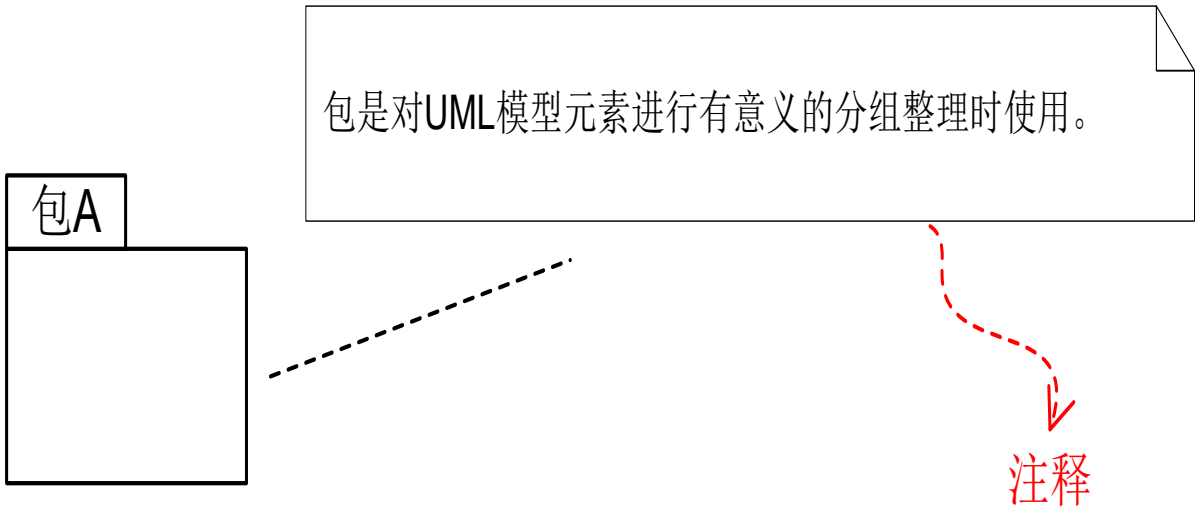
- 当两个包内的元素具有互相使用的关系时，两个包之间具有双向依赖关系。



包之间的双向依赖关系会造成包维护修改上的循环影响，在划分包的时候应尽可能避免。

注释（Note）

- 注释（Note），是对UML的模型元素进行解释，约束，或者添加补充信息时使用。
- 注释（Note）和包（package）在UML1.5中都是作为UML的公用机制出现的，顾名思义，即可用于UML的所有模型元素上。
- 注释（Note）用带折角的矩形框来表示。



11.2.3设计包的原则

◆在建立包时，应把概念上和语义上相近的模型元素纳入一个包。好的设计要求体现高内聚、低耦合的特性。在设计包时，应遵循以下一些原则：

- ■重用等价原则
- ■共同闭包原则
- ■共同重用原则
- ■非循环依赖原则

◆重用等价原则(Reuse Equivalency Principle, REP) 指的是把类放入包中时，应考虑把包作为可重用的单元。

◆这种设计原则和用户的使用心理有关，对于可重用的类，其开发可能比较快，开发人员会不断地推出这些可重用类的升级版本，但对于可重用类的使用者来说，不会随着可重用类的每次升级而修改自己的系统，不过，在需要升级的时候又会要求很容易地用新版本的可重用类替换旧版本的可重用类，因此设计包的一个原则是把类放在包中时要方便重用，方便对这个包的各个版本的管理。

◆共同闭包原则(Common Closure Principle, CCP) 指的是把那些需要同时改变的类放在一个包中。

◆例如，如果一个类的行为和/或结构的改变要求另一个类作相应的改变，则这两个类应放在一个包中；或者在删除了一个类后，另一个类变成多余的，则这两个类应放在一个包中；或者两个类之间有大量的消息发送，则这两个类也应放在一个包中。

◆共同闭包原则就是要提高包的内聚性、降低包与包之间的耦合度，希望在改动或升级一个包的时候要尽量少影响别的包。

◆共同重用原则(Common Reuse Principle, CRP) 指的是不会一起使用的类不要放在同一包中。

◆这个原则和包的依赖关系有关。如果元素A依赖于包P中的某个元素，则表示A会依赖于P中的所有元素。也就是说，包P中的任何一个元素做了修改，则都要检查元素A是否还能使用包P，即便所修改的元素和A完全没有关系。

◆修改了一个对用户实际上毫无影响的类，却逼得用户不得不重新检查是否还可以同样方式使用新的包是不合理的。

说明：

◆重用等价原则、共同闭包原则、共同重用原则

这3个原则事实上是相互排斥的，不可能同时被满足，它们是从不同使用者的角度提出的，重用等价原则和共同重用原则是从重用人员的角度考虑的，而共同闭包原则是从维护人员的角度考虑的。

◆共同闭包原则希望包越大越好，而共同重用原则却要求包越小越好。

◆在开发的早期，可以共同闭包原则为主，而当系统稳定后，可以对包做一些重构，这时要以重用等价原则和共同重用原则为主。

◆非循环依赖原则(Acyclic Dependencies Principle, ADP) 指的是包之间的依赖关系不要形成循环。也就是说不要有包A依赖于包B，包B依赖于包C，而包C又依赖于包A这样的情况出现，如果确实无法避免出现包之间的循环依赖，则可以把这些有循环依赖关系的包放在一个更大的包中，以消除这种循环依赖关系。

◆非循环依赖原则(ADP: The Acyclic-Dependencies Principle)

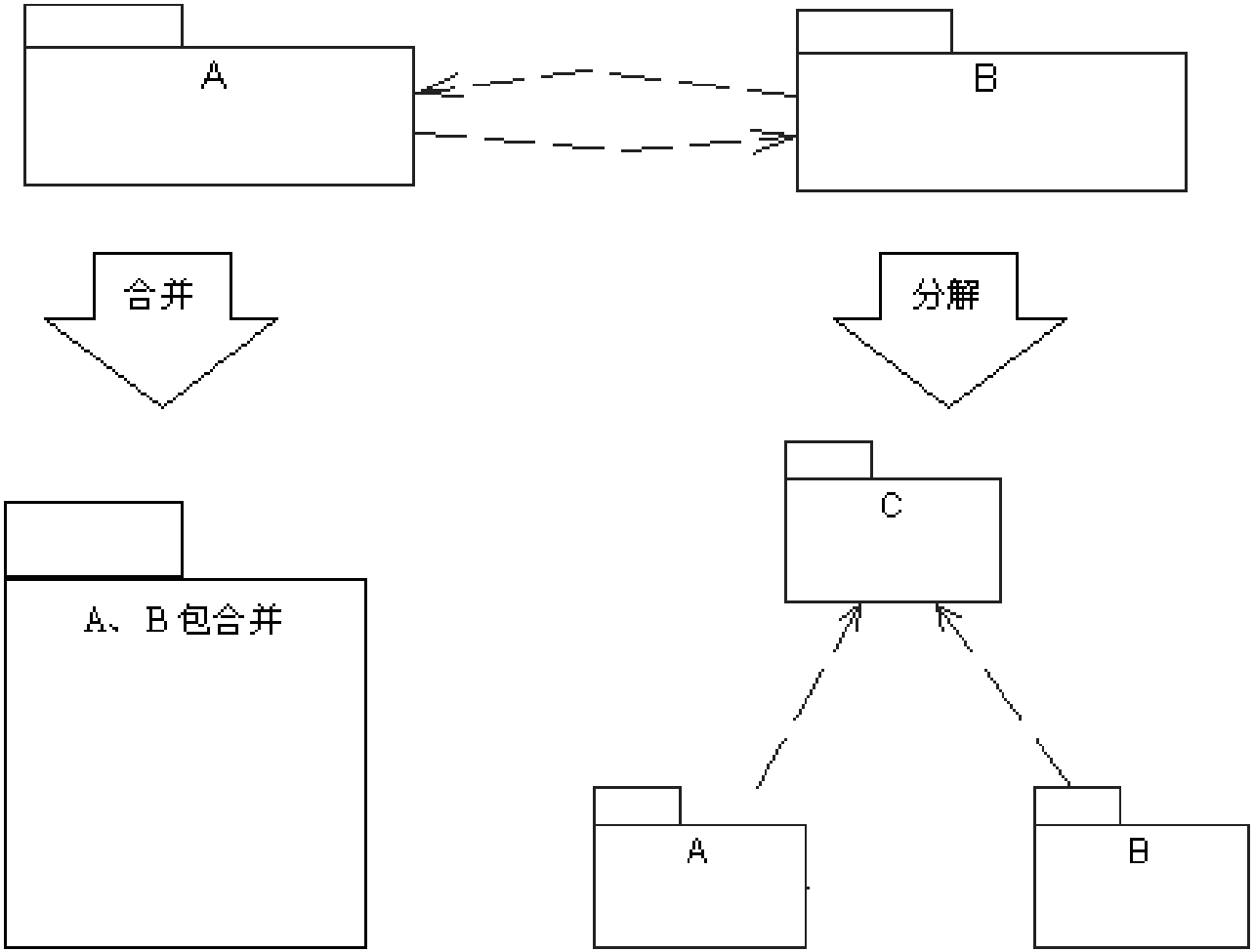
■包之间的依赖关系不要形成循环：即不要出现A依赖B，B又依赖于A的现象，或者多个包形成循环依赖。

消除包的循环依赖

应该尽量避免包模型中的循环依赖。如果包A以某种方式依赖包B，并且包B以某种方式依赖包A，就应该合并这两个包，这是消除循环依赖非常有效的方法。

但是，经常起作用的、更好的方法是，从A,B两个包中提取公共元素，把它们封装为第三个包C。消除循环包的过程是一个多次迭代的过程。

包的设计原则



消除循环依赖的两种方法

包图建模风格

◆建模风格1：使包具有内聚性。

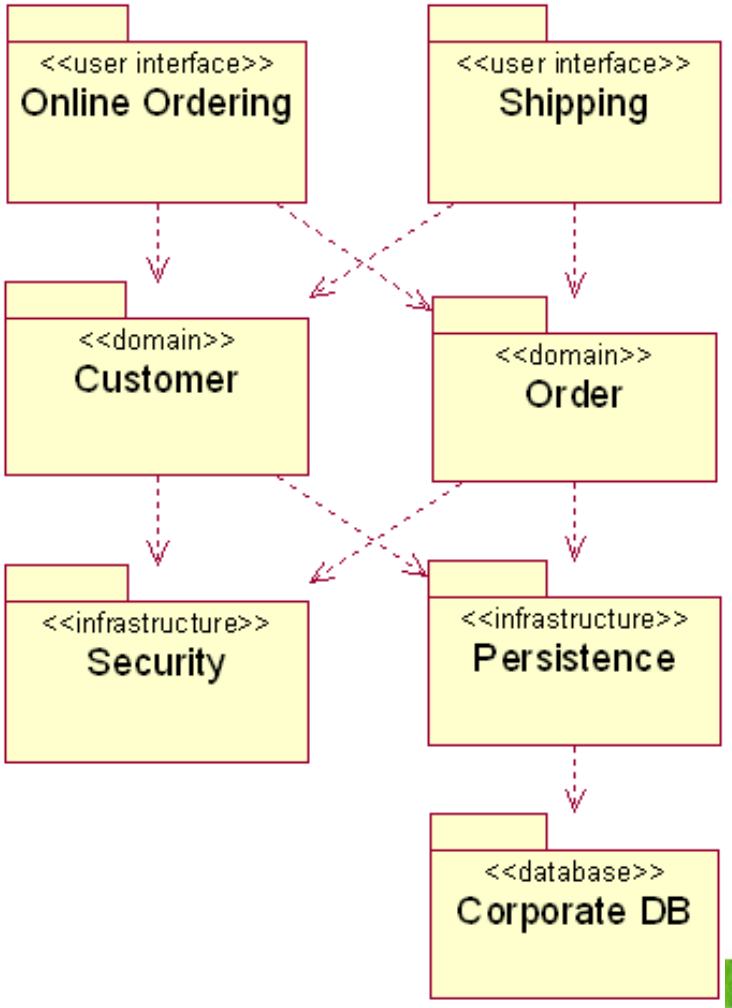
■放在包中的任何东西，当和包中的其余内容一起考虑时应该有意义。

■一个好的判断包是否具有内聚性的方法是看能否给包起一个简短的、描述性的名字，如果不能，则很可能把几个不相关的东西放在一个包中了。

◆建模风格2：垂直分层类包图。

■包位置的放置反映了系统体系结构的逻辑分层。按照惯例，一般是采用自上而下的方式画分层结构。

– 例：用户界面和领域类交互，而领域类又会使用基础结构类，某些基础结构类又会存取数据库。



◆建模风格3：在包上用版型指明体系结构的层次。

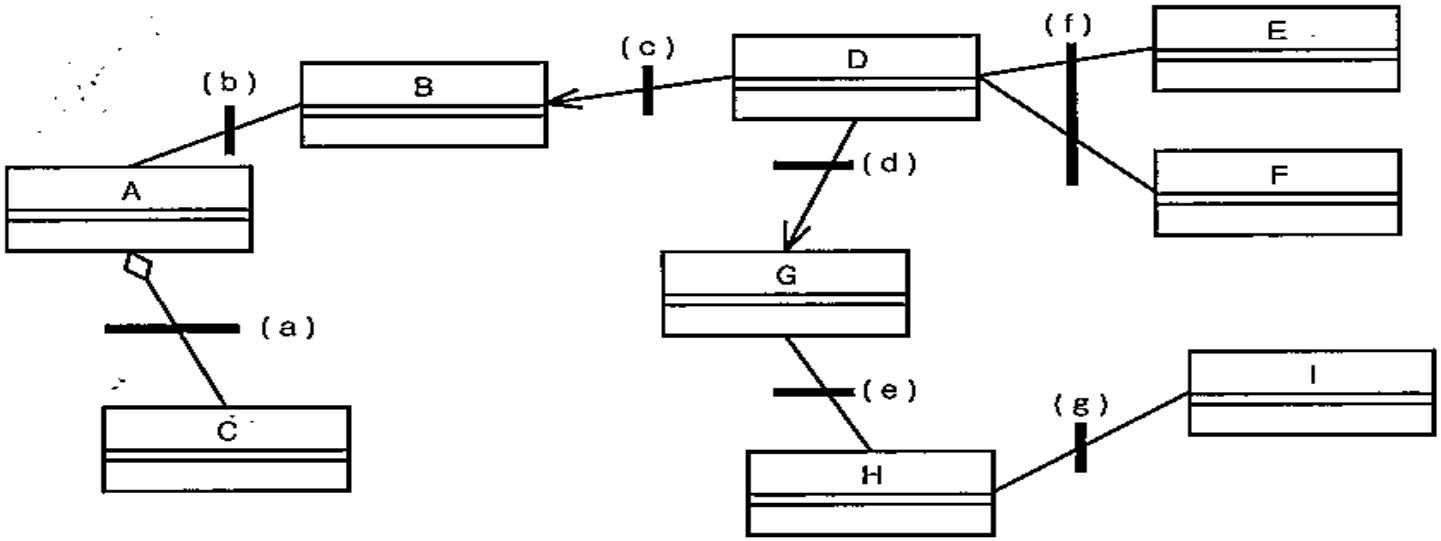
- ■把设计按照体系结构的层次进行组织是非常常见的，比如用户界面层、业务/领域层、持久/数据层、基础设施/系统层等进行组织。
- ■例：可以用版型(sterotype) <<user interface>>、<<domain>>、<<infrastructure>>和<<database>>等表示不同层的包。

总结

- 包图是用来对UML模型元素进行有意义的分组整理时使用，可提高模型的可读性和可维护性。
- 注释是对UML模型元素进行解释，添加补充信息时使用。同包一样，属于UML中的共有机制，可使用在任何UML元素上。
- 包和包之间的关系具有依赖关系和泛化关系两种。取决于包内元素之间的关系。
- 在进行分包的时候，要尽量避免包之间的双向依赖关系。

例题1

要将下面的类图分割成3个包。从下列选项中选择合适的分割位置。

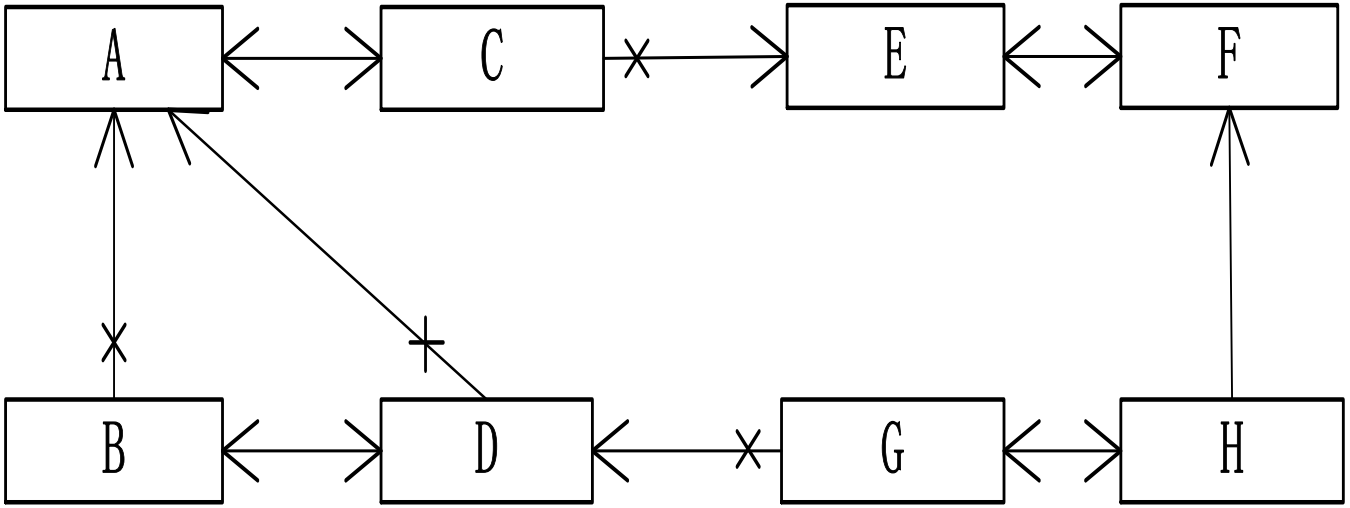


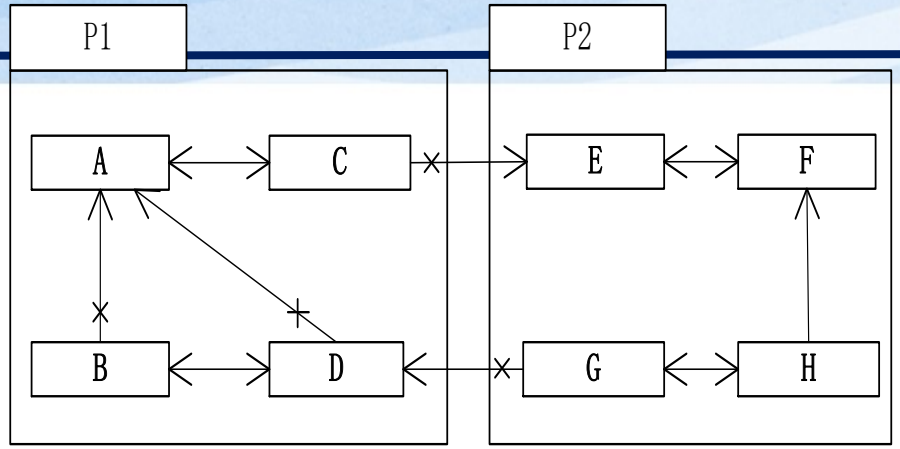
(1) a　(2) b　(3) c　(4) d　(5) e　(6) f　(7) g

分割位置：(c)(d)
从(c)(d)分割，产生的依赖数量最小。

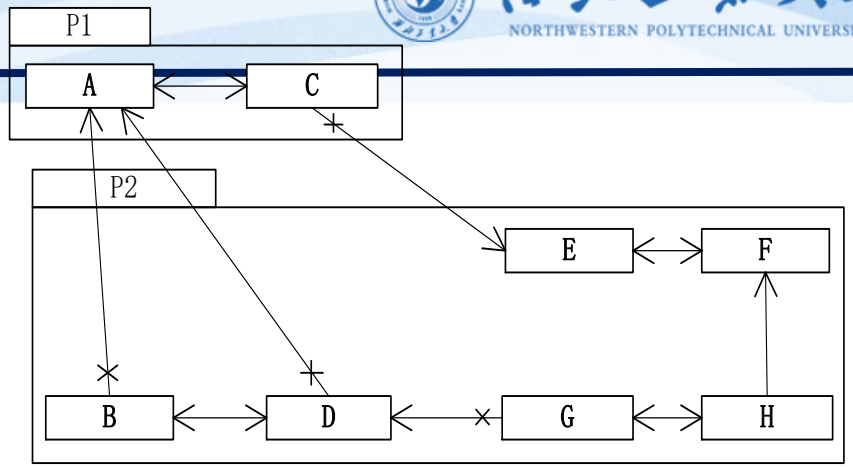
例题2

考虑对下面的类图分割包。请从选项中选出2个不恰当的选项

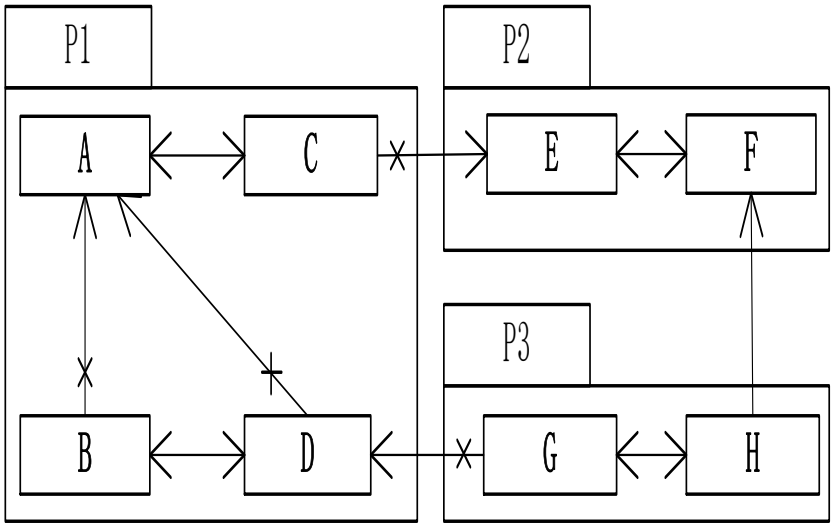




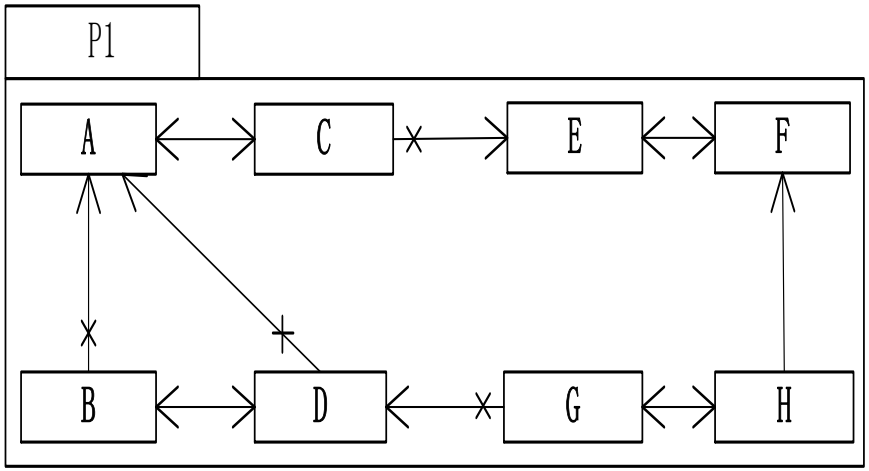
A



B



C



D

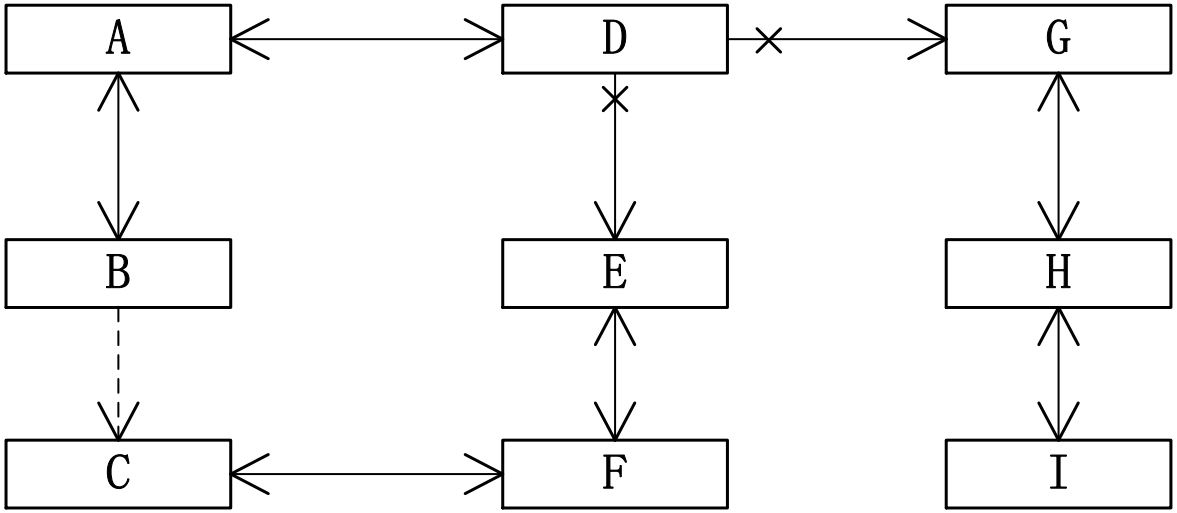
例题2

原则：
分割包的时候，注意以下几点。
1)包与包之间的依赖关系尽可能少
2)避免双向依赖关系
3)包内的类不能过多（100个以下）

A、B中，包之间的依赖关系是双向的，特别是B的依赖关系比A还多，所以是不恰当的。C没有双向依赖关系，包之间的依赖关系只有一个，所以是恰当的。D只有一个包。只有8个类，可以说不必非要分割包。所以，D也是恰当的。

例题3

把下面类图划分到3个包中。请选出最适合的类分组的组合 。



A. （A类、B类、C类、D类）、（E类、F类）、（G类、H类、I类）
B. （A类、B类、D类）、（C类、E类、F类）、（G类、H类、I类）
C. （A类、B类、D类、E类）、（C类、F类）、（G类、H类、I类）
D. （A类、B类、C类）、（D类、E类、F类）、（G类、H类、I类）

例题3

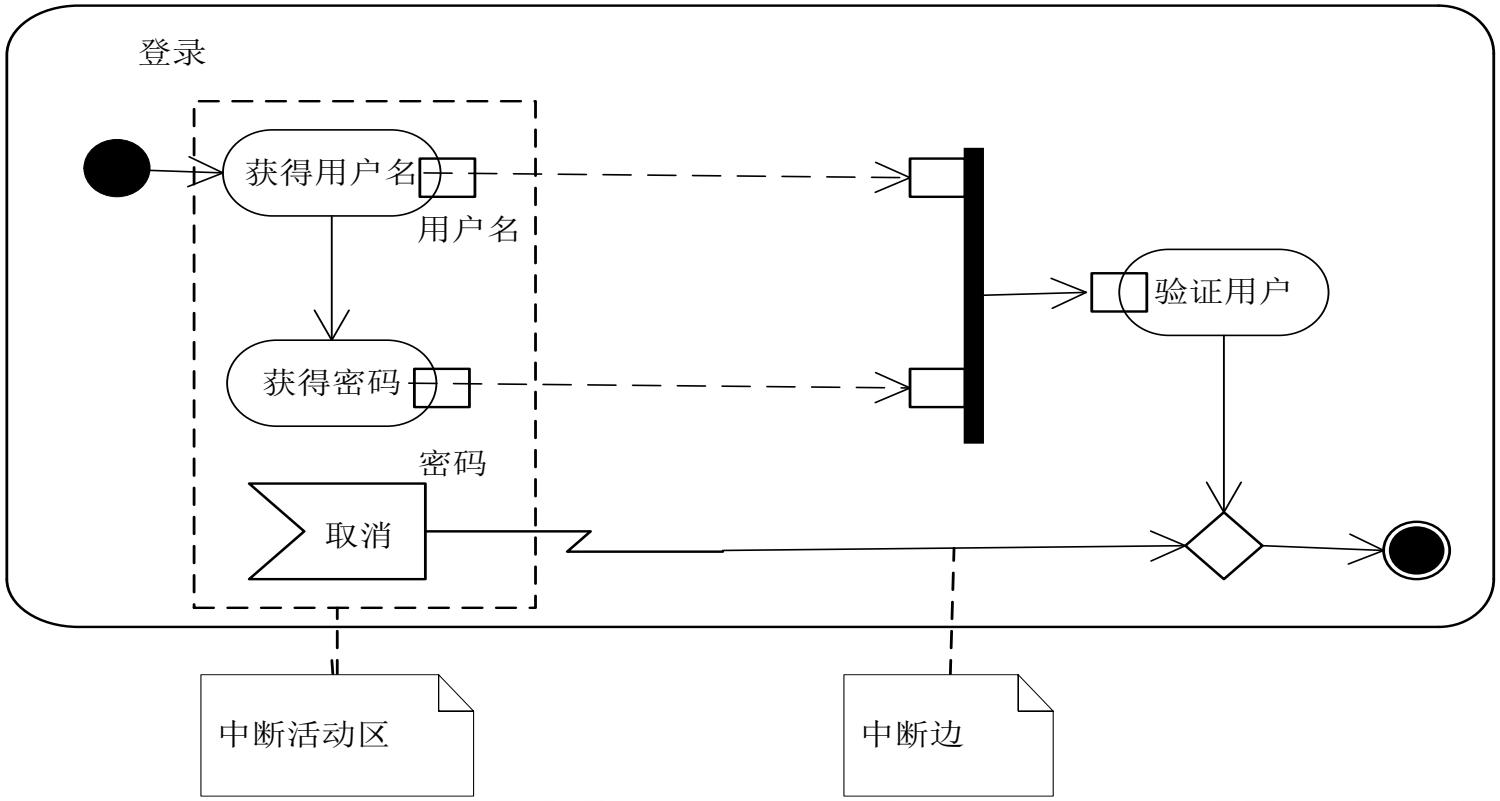
原则：

分割包时，应该尽量从类之间的关系薄弱的地方分开。在关联数少、单向关联的地方划分。

答案:B

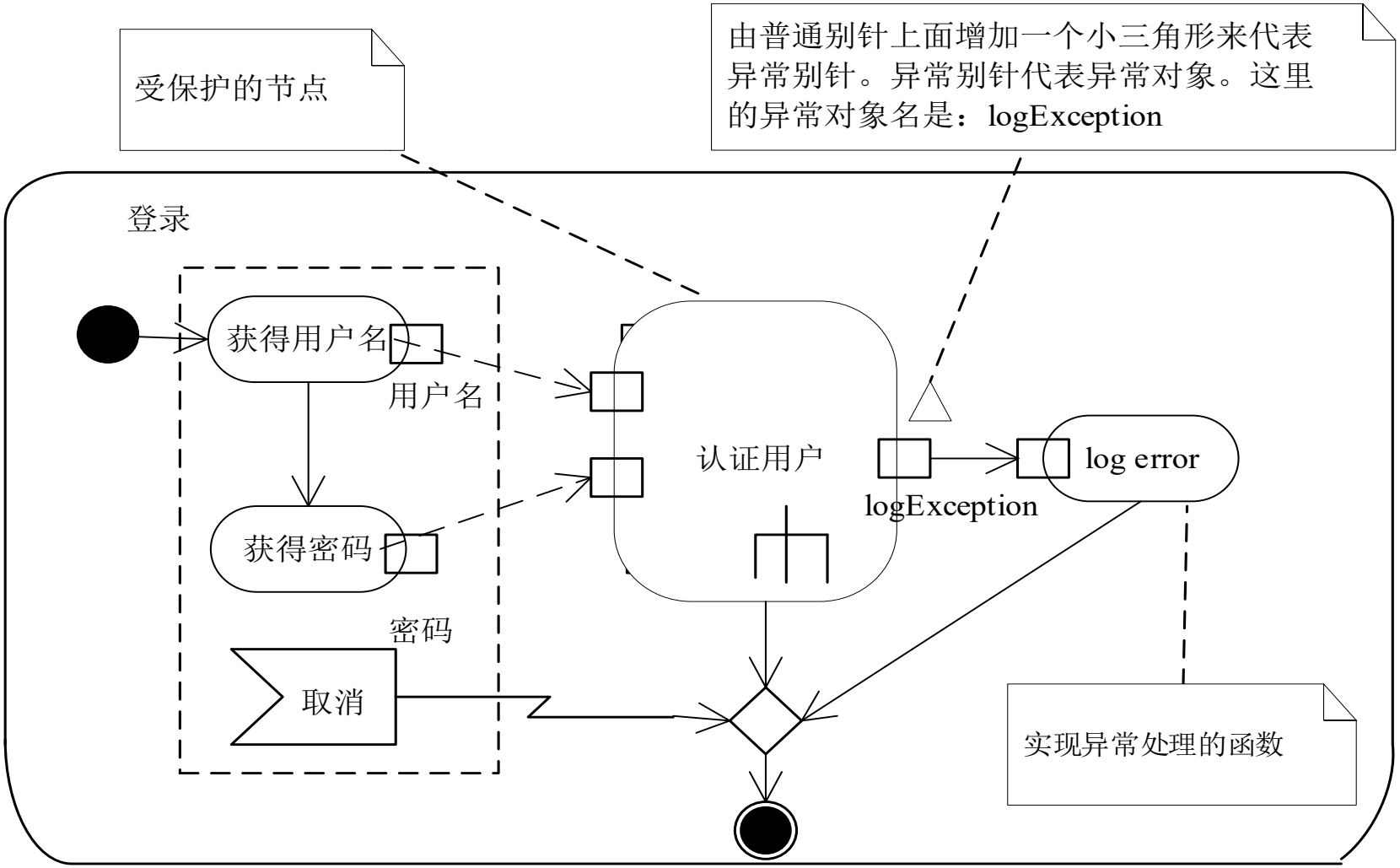
展示中断的活动图*

◆图中，我们把三个活动封闭在虚线框中，这个虚线框就是中断区。中断边用齿形箭头表示。当控制焦点在中断活动区时，当”取消”活动收到一个中断事件后，中断区中的三个活动都会停止执行，控制焦点转向中断边。



展示异常的活动图*

◆如图所示，用UML图展示了计算机的异常处理流程。



展示异常的活动图

展示信号的活动图*

- 信号用来展示两个对象之间进行的异步通信。当一个对象接收到一个信号时，将触发信号事件。
- 在活动图中包括3种信号元素，分别是发送信号、接收信号和时间信号。表示方法如图所示



在活动图中标识信号*

◆如图所示，展示了手机卡验证的信号图。

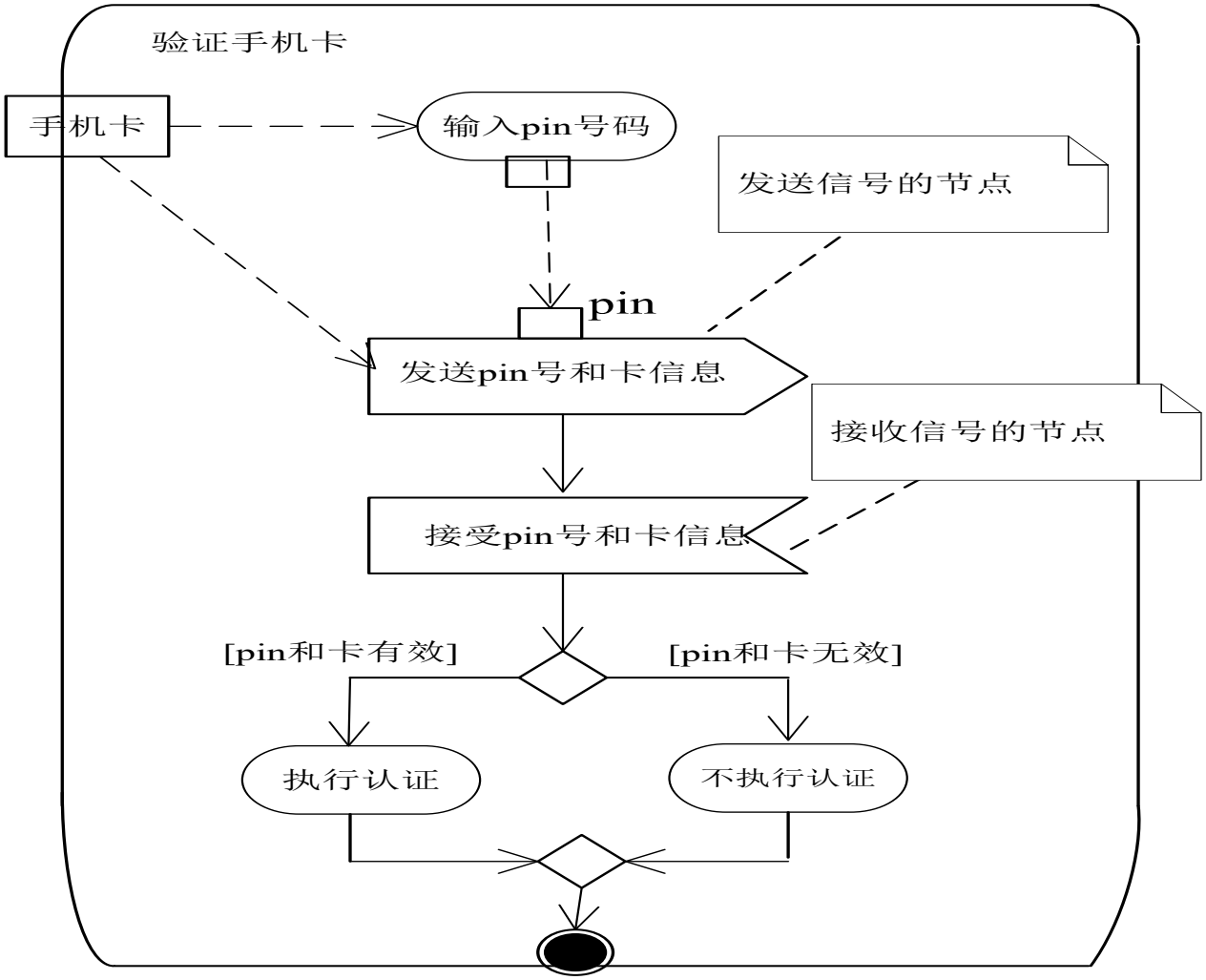
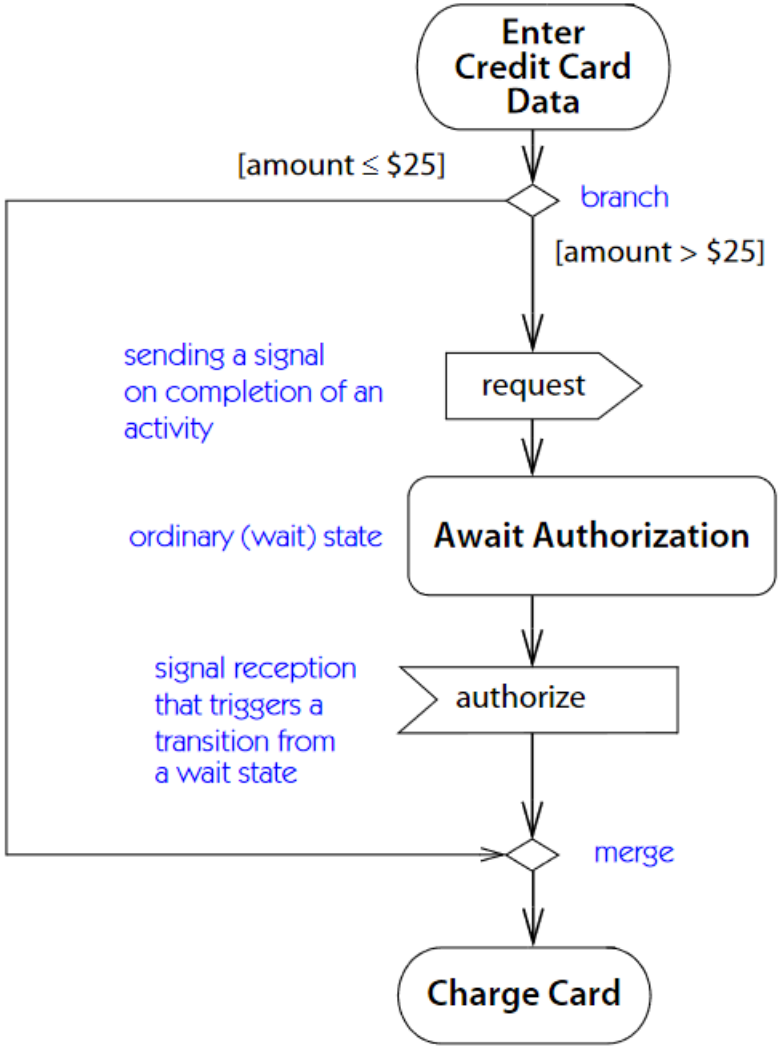


图8-18 手机pin验证

收发信号的更多例子*



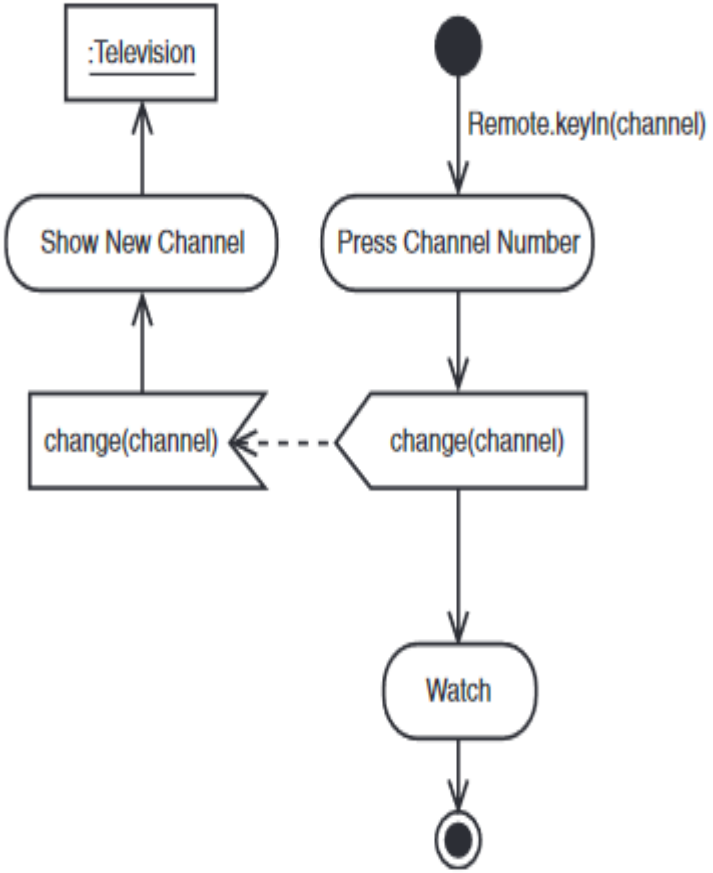
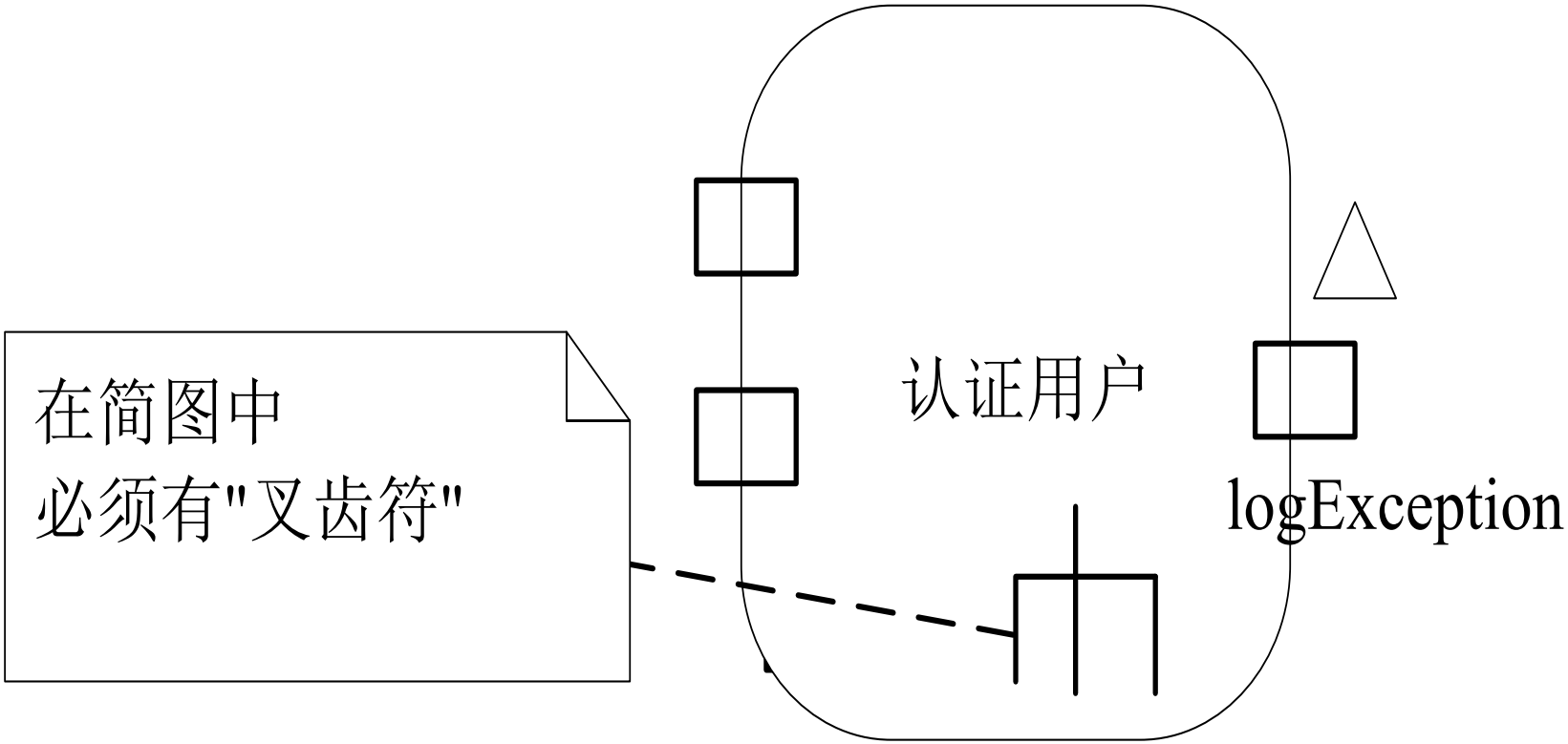


Figure 14-183. *Activity diagram showing sending and receiving of signals*

展示嵌套活动图*

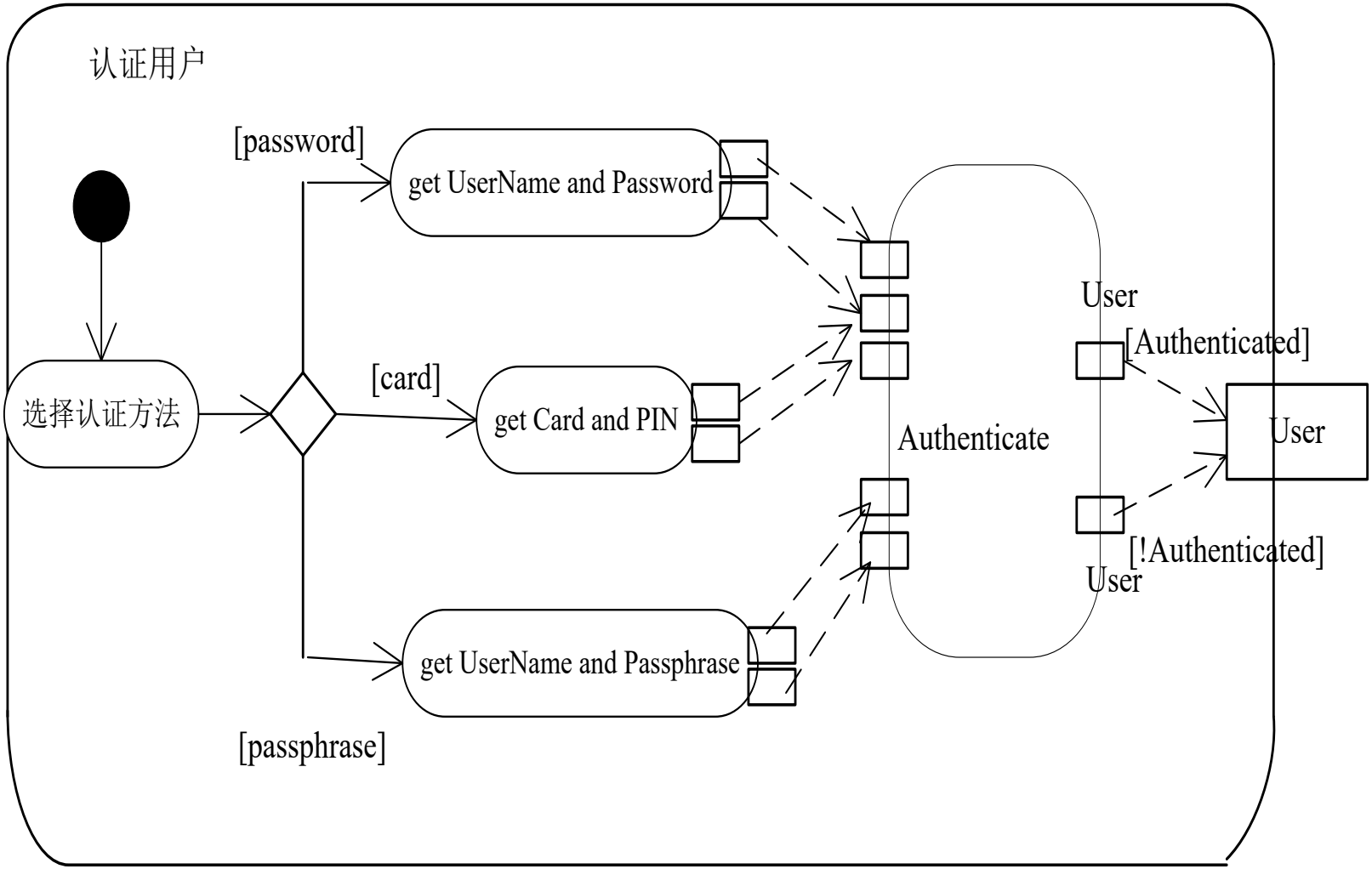
- 当一个活动图很复杂，则可以把其中的一组相关活动看作是一个子活动图，这时在绘制主活动图时，可用子活动图的简图代替子活动图。

◆子活动图”认证用户”其简图如图所示。



认证用户的简图

◆如图所示是一个嵌套活动图，其中的节点”认证用户”是一个子活动图的简图。



认证用户